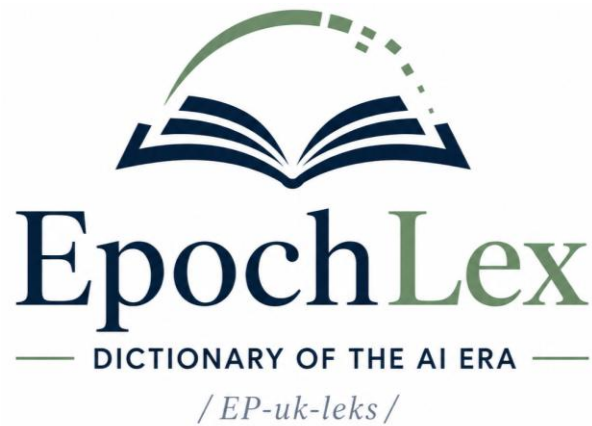




DESIGNED RESEARCH CASE STUDY

THE EPOCHLEX EXPERIMENT

Building a Dictionary of the AI Era through Human-Directed, AI-Assisted Development

A case study of product development, research, ambiguity, validation, and evolving AI workflows

Observation Period

August 28–September 17, 2026

Report Snapshot

September 18, 2026

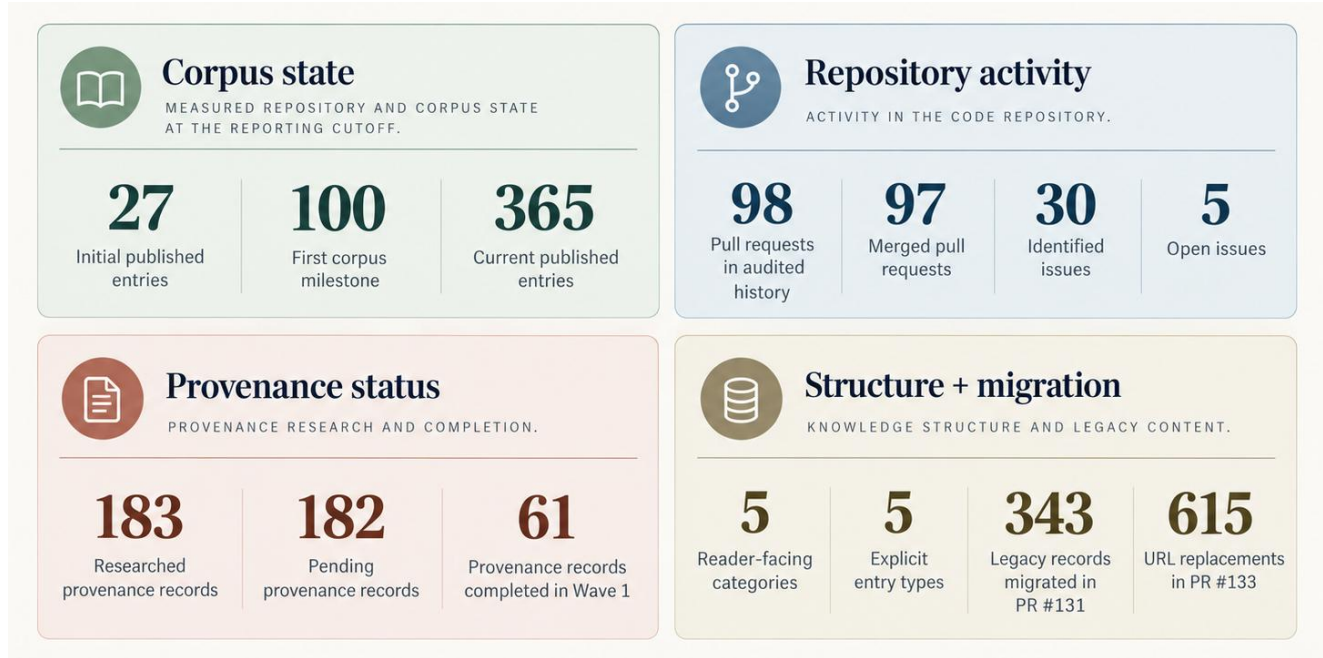
EpochLex

Dictionary of the AI Era

REPORT SNAPSHOT

Key Metrics at a Glance

The September 18, 2026 snapshot provides a concise view of the scale of the first observation period. These measures describe different forms of activity and should not be combined into a single productivity score.



REPORT MAP

Contents

Key Metrics at a Glance	2
Executive Summary.....	8
1. Introduction, Scope, and Methodology.....	10
2. Purpose of the Case Study.....	10
3. Questions Examined.....	11
4. Observation Period and Evidence Base.....	11
5. Development Environment.....	12
6. Quantitative Evidence.....	13
7. What This Report Does Not Claim.....	13
8. A Note on Ambiguity	14
Part II — Reconstructing the Experiment	15
9. From a Term List to a Working Product.....	15
10. Rapid Expansion and the First Regression Cycle	15
11. Quality Assurance Becomes Part of the Product.....	16
12. The AILex to EpochLex Migration.....	17
13. The Taxonomy Expands Beyond Terms.....	17
14. The PWA and the Boundary of the Static Architecture.....	18
15. From a Private Project to an Open-Source Operation.....	18
16. Making the Research Pipeline Persistent	19
17. Scaling the Corpus Changes the Bottleneck.....	19
18. Word of the Day and the Question of Architectural Cost.....	20
19. The PWA Becomes an Intentional Product Experience.....	20
20. Introducing Codex Into an Existing Workflow	21
21. Maintenance Work Becomes More Explicit.....	22
22. What Changed During the First Three Weeks	22
Part III — Case Studies and Findings.....	23
23. Case Study One: From 27 Terms to a Working Reference Product.....	23
Context	23
What Happened.....	23
What Changed Afterward.....	24
24. Case Study Two: Provenance Changed the Nature of the Project.....	24
Context	24
What Happened.....	25

What Changed Afterward.....	25
25. Case Study Three: Mobile Development and the Difference Between Functional and Correct	25
Context	25
What Happened.....	26
What Changed Afterward.....	26
26. Case Study Four: The AILex to EpochLex Rename	26
Context	26
What Happened.....	27
What Changed Afterward.....	27
27. Case Study Five: Open Source Required More Than Publishing the Code.....	28
Context	28
What Happened.....	28
What Changed Afterward.....	28
28. Case Study Six: Rapid Corpus Growth and the Creation of Research Debt	29
Context	29
What Happened.....	29
What Changed Afterward.....	29
29. Case Study Seven: Word of the Day, Push Notifications, and Architectural Restraint	30
Context	30
What Happened.....	30
What Changed Afterward.....	31
30. Case Study Eight: A Stale Pull Request and the Introduction of Codex.....	31
Context	31
What Happened.....	31
What Changed Afterward.....	32
31. Case Study Nine: Strong Automated Validation Did Not Eliminate Human Testing.....	32
Context	32
What Happened.....	32
What Changed Afterward.....	32
32. Case Study Ten: Ambiguity as Part of the Experiment.....	33
Context	33
What Happened.....	33
When Ambiguity Becomes a Liability	33
When Ambiguity May Remain Useful.....	34
What Changed Afterward.....	34
33. Cross-Case Findings	35

34. A Developing Interpretation of “Vibe Coding”	36
35. The Role of Human Judgment	36
36. The Emerging Development Model	36
37. Interim Finding	37
Part IV — Quantitative Analysis and Experimental Measures.....	38
38. Why Quantitative Measures Matter	38
39. Repository Activity During the Observation Period	38
Repository Snapshot	38
40. Development Was Highly Concentrated in Bursts.....	39
Major Activity Bursts.....	39
41. Corpus Growth	40
42. Provenance as a Quantifiable Bottleneck	40
43. Pull Requests as a Measure of Iteration, Not Productivity	41
44. Rework and Follow-Up Work	42
45. Human-Detected Versus Automated Findings	42
46. The Toolchain Timeline.....	43
Known Toolchain Phases.....	43
47. Model Changes Are Part of the Experiment.....	43
48. Token Utilization	44
49. Token Counts Are Not Productivity Scores.....	45
50. Human Effort Is the Largest Missing Quantitative Measure.....	45
51. Measuring the Next Phase Beginning With Codex.....	45
52. Tracking Ambiguity as Data.....	46
53. Candidate Metrics for the Continuing Experiment	47
54. What Can Be Quantified Now	48
55. What the Numbers Tell Us	48
Part V — Discussion, Lessons, and Implications.....	48
56. The Experiment Was Not Really About Whether AI Could Write Code.....	48
57. AI Reduced the Cost of Trying an Idea	49
58. Faster Implementation Changes the Economics of Iteration	49
59. The Hammer and the Nail Gun	50
60. Established Development Practices Did Not Disappear.....	50
61. Repository Context Became a Form of Organizational Memory.....	50
62. Documentation Is Part of the AI Control System.....	51
63. AI Can Accelerate Technical Debt as Easily as Features.....	51
64. Human Judgment Became the Scarcer Resource.....	51

65. Human Review Is More Than Error Checking	52
66. Ambiguity Is Shared Between the Human and the AI	53
67. The Ability to Surface Ambiguity May Become a Model Capability Worth Measuring	53
68. Product Responsibility Limits Experimental Freedom.....	53
69. Architecture Became a Decision Framework.....	54
70. Simplicity Has Ongoing Value in AI-Assisted Systems	54
71. AI-Assisted Research Has Different Risks From AI-Assisted Coding.....	54
72. The Meaning of “AI-Built” Requires Precision.....	55
73. Tool Changes Complicate Cause and Effect.....	55
74. The Work Is Shifting Rather Than Disappearing.....	55
75. Implications for Professional Practice	56
76. Process Knowledge May Outlast Tool Knowledge.....	56
77. Vibe Coding and Professional Practice Are Not Necessarily Opposites.....	56
78. What the Experiment Suggests So Far	57
79. A More Mature Statement of the Experiment	57
80. The Central Lesson	57
Part VI — Conclusion, Limitations, and the Continuing Experiment	58
81. The September 18 Snapshot.....	58
82. What the First Phase Established.....	58
83. Limitations of the First Report	59
84. Limits on Generalization	59
85. Codex Marks a New Observation Period.....	59
86. Measurement Plan for the Next Phase	60
87. Token Utilization Status	60
88. Model Timeline Status.....	60
89. Contributors and the Expanding Experimental Environment.....	61
90. Ambiguity Should Continue to Be Observed Deliberately	61
91. EpochLex as a Living Experiment	61
92. Conclusion.....	62
Appendix A — Evidence Register	64
Evidence Classes	64
Core Evidence Register	64
Evidence-to-Finding Map.....	66
Appendix B — Repository Classification Framework	68
Primary Classification.....	68
Completed PR Classification	68

Secondary Tags.....	69
Appendix C — Model and Toolchain Timeline.....	70
Interpretive Boundary.....	70
Appendix D — Token Utilization Reconstruction	70
Stage 1 — Conversation Identification.....	70
Stage 2 — Visible Token Count.....	70
Stage 3 — Context Reconstruction.....	71
Stage 4 — Direct Analytics	71
Stage 5 — Reporting.....	71
Appendix E — Next-Phase Measurement Framework.....	71
Unexpected Outcome Classification	72
Appendix F — Report Terminology	73
AI-Assisted Development.....	73
Human-Directed.....	73
Human-Reviewed	73
Vibe Coding	73
Agentic Development.....	73
Ambiguity.....	73
Corrective Rework	73
Iterative Refinement	73
Provenance.....	73
Appendix G — Source Notes and Methodology Details	74
Source Hierarchy	74
Evidence Handling	74
Citation Convention.....	74
Measurement Labels	74

Executive Summary

EpochLex is an open-source, searchable reference work for terminology of the AI era. The project began on August 28, 2026, as a relatively small dictionary concept and developed into a 365-entry public reference work with structured provenance—source, origin, and history records—dedicated term pages, pronunciation, responsive browsing, related-term discovery, a Progressive Web App (PWA), Word of the Day, contributor infrastructure, and a documented editorial and architectural model.

The project also became an experiment in AI-assisted development. ChatGPT was used extensively alongside GitHub and repository-based development during the initial phase, assisting with product exploration, implementation, debugging, documentation, research organization, quality assurance, and editorial support. On September 17, Codex was added to the existing workflow as a more repository-oriented implementation agent. The development process remained human-directed and human-reviewed throughout.

This report covers the first major observation period, from August 28 through September 17, 2026, using a September 18 repository snapshot as the baseline. It draws primarily from the public GitHub repository, pull requests (reviewable change sets), issues, source-controlled documentation, structured datasets, and development conversations. The repository state at the cutoff is treated as the source of truth for what EpochLex was at that point, while earlier conversations help explain intent, rejected alternatives, ambiguity, and the reasoning behind changes.

The central finding is that AI materially reduced the distance between product intent and working implementation. Features, migrations, content expansions, and revisions could be produced quickly enough that working software often became part of the design and requirements process. That speed did not remove the need for planning or engineering discipline. As the project grew, architecture, quality assurance, source control, documentation, validation, editorial standards, and human review became more explicit, not less.

Several failures were just as informative as the successful work. Mobile implementations could function technically while still missing the intended experience. Browser and PWA caches could preserve outdated behavior even when the repository contained the correct source. Pronunciation logic could be implemented correctly while still sounding wrong on an actual device. Rapid corpus expansion increased publication throughput but created a large downstream provenance-research workload. Together, these cases show that AI-assisted development can accelerate useful work and incomplete assumptions at the same time.

Another recurring lesson concerns ambiguity. An unexpected implementation is not always evidence that the AI wrote bad code. In several instances, the implementation exposed a requirement or product assumption that the human had never stated explicitly. EpochLex therefore distinguishes between ambiguity that must be resolved because it affects security, privacy, data integrity, architecture, core behavior, or materially misleading user experiences and lower-risk ambiguity that may remain open briefly when observing it can teach something useful about AI behavior.

The repository contained 98 pull requests in the audited history, of which 97 were merged. The published corpus grew from 27 entries to 365. At the September 18 snapshot, 183 provenance records had completed an initial human-reviewed research pass and 182 remained pending. These numbers show the scale and pace of the work,

but they do not establish a productivity multiplier, cost savings, or superiority over conventional software development because the experiment did not include a matched control project or a reliable historical labor record.

What changed most clearly was the rate at which ideas could become working implementations, not the need for judgment. Increased production capacity shifted more attention toward deciding what should be built, defining what must remain unchanged, evaluating evidence, detecting ambiguity, validating actual behavior, and knowing when to stop. The introduction of Codex marks the beginning of a second observation period in which those questions can be tracked more deliberately.

The EpochLex Experiment

A Case Study in Human-Directed, AI-Assisted Development

EpochLex — Dictionary of the AI Era

Observation period: August 28–September 17, 2026

Repository snapshot for this report: September 18, 2026

1. Introduction, Scope, and Methodology

EpochLex began as a personal effort to collect and understand terminology emerging around artificial intelligence. The immediate spark was the term *Meat Proxy*, encountered in a post on X, but the larger issue was the pace at which AI was generating technical language, workplace terminology, research concepts, governance language, and cultural slang. A simple list no longer seemed sufficient, so the idea grew into a searchable, structured reference work.

The project did not initially begin as an experiment in AI-assisted software development. The dictionary came first. Early work focused on questions such as what terms belonged in the collection, how they should be categorized, how definitions should be written, and how readers should be able to search and explore the material. The idea of using artificial intelligence to build the dictionary emerged only after the product concept had already begun to take shape.

That second idea changed the nature of the project. EpochLex would continue to function as a real public reference product, but its development process would also become an experiment in how far a human-directed AI workflow could support design, construction, research, documentation, testing, maintenance, and product evolution. The experiment was never only about whether AI could produce working HTML, CSS, JavaScript, or JSON. It increasingly asked what happens when AI participates across a much broader product lifecycle.

The development model that emerged is described in the repository as “Vibe coded · Human-directed · AI-assisted · Human-reviewed.” In this context, “vibe coded” does not mean accepting generated output without review or allowing an AI system to determine independently what the project should become. It describes a conversational development process in which product intent, requirements, constraints, problems, and feedback are communicated to AI systems in natural language, while those systems assist with implementation and analysis. Human judgment remains responsible for deciding what EpochLex should be, whether an implementation is acceptable, whether evidence supports an editorial claim, and what ultimately reaches the public project.

That distinction makes EpochLex useful as more than a code-generation example. The project was not developed autonomously, but AI was also not limited to occasional autocomplete or isolated code snippets. AI assistance became materially involved in product exploration, information architecture, interface design, code generation, debugging, documentation, research support, source discovery, editorial drafting, quality assurance, and repository maintenance. The result is a record of AI working inside an evolving product-development system rather than a collection of disconnected prompts.

2. Purpose of the Case Study

This report documents what happened during the EpochLex experiment and separates what the record shows from broader claims about AI-assisted development. Much of the work is preserved in a public GitHub repository through source files, pull requests, issues, documentation, datasets, fixes, and revisions. Development conversations

add another layer by preserving product intent, rejected alternatives, design reactions, assumptions, and decisions that are often invisible in repository history alone.

This is not primarily a success story about what AI can build. Successful implementations matter, but so do regressions, stale assumptions, incomplete requirements, browser-specific behavior, research backlogs, documentation drift, abandoned approaches, and changes in working method. In several cases, the most useful lesson came from an implementation that technically worked but exposed a problem that had not been recognized before.

EpochLex also has two roles at once: it is a public reference product and an ongoing experiment in AI-assisted development. Those roles usually align, but not always. A production product benefits from resolving consequential ambiguity and making behavior more predictable. An experiment may sometimes benefit from leaving low-risk uncertainty visible long enough to see how another model, prompt, context, or workflow responds.

For that reason, the report asks not only whether AI-assisted work succeeded or failed, but why. A problem may come from incorrect code, missing repository context, a mistaken AI assumption, conflicting requirements, an unstated human expectation, or a decision that no one had realized needed to be made. Distinguishing among those causes is more useful than treating every unexpected result as a generic “AI failure.”

3. Questions Examined

The report is organized around several related questions. The first is about capability: how much of a real public product lifecycle can AI materially assist while a human continues to provide direction and judgment? EpochLex provides examples across product design, software implementation, editorial work, research organization, debugging, documentation, quality assurance, and maintenance rather than only initial code generation.

A second question concerns process. As EpochLex grew, did AI assistance reduce the need for conventional development disciplines, or did the project become increasingly dependent on architecture, source control, testing, documentation, issue tracking, review, and clearly defined responsibilities? The repository history allows this question to be examined over time because the development process became substantially more formalized during the observation period.

A third question concerns throughput and bottlenecks. AI-assisted workflows made it possible to create and revise substantial amounts of software, documentation, and dictionary content in a relatively short period. The report looks at whether that extra production capacity removed work or simply moved the constraint elsewhere, particularly into provenance research, validation, maintenance, and human review.

The report also examines ambiguity. AI-generated code can be technically reasonable while still implementing an assumption that was never explicitly intended. EpochLex therefore provides a way to examine when ambiguity should be turned into a requirement, constraint, test, or documented decision and when low-risk ambiguity may remain open briefly because observing it provides useful information about AI reasoning and consistency.

Finally, the development environment itself changed during the observation period. The maintainer workflow initially combined ChatGPT with GitHub and repository-based development. Beginning September 17, 2026, Codex was added to that existing workflow: ChatGPT continued to support product thinking, research, planning, and design, while Codex began handling more bounded repository implementation work. Outside contributors may use different AI systems, conventional development environments, or entirely different workflows, so contributor tooling is recorded when known rather than assumed to match the maintainer workflow.

4. Observation Period and Evidence Base

The primary observation period for this report begins on August 28, 2026, when the first working Dictionary of the AI Era repository implementation was created, and runs through September 17, 2026, when Codex was formally introduced into the maintainer workflow. The repository snapshot used for this edition is September 18, 2026. For repository-state claims, the frozen baseline is main immediately after PR #133, merge commit 4b31946, the final

merge in the audited observation set. Later work may be referenced where necessary to clarify an issue, but it is not silently incorporated into measurements defined by this initial observation window.

The strongest evidence is the repository itself. The frozen snapshot baseline governs claims about the September 18 report state, while current main remains the source of truth for present-day EpochLex outside this historical cutoff. Pull requests and issues provide the record needed to reconstruct how the snapshot developed, including implementations that were later corrected, superseded, or deliberately left unmerged.

Development conversations provide a second evidence layer. These records are particularly useful for understanding why changes occurred, because source control generally records what changed but not every judgment that led to the change. Conversations preserve examples of interface concepts being rejected, brand directions being reconsidered, assumptions being exposed through testing, features being deferred because of architectural consequences, and development workflows being deliberately revised.

The final evidence layer is interpretation. The report compares repository history, current documentation, development conversations, and observable outcomes to explain what the project learned. Those interpretations describe the EpochLex case; they are not presented as universal rules for AI-assisted software development.

When these evidence sources conflict, the frozen snapshot baseline takes precedence for claims about the September 18 report state, while current main takes precedence for claims about EpochLex as it exists after that cutoff. Older conversations and project materials remain useful historical evidence, but they are not used to overwrite decisions subsequently established in the repository. Material contradictions are retained as evidence of project evolution rather than silently reconciled.

5. Development Environment

For most of the initial experiment, the maintainer workflow can be described as ChatGPT + GitHub/repository development. ChatGPT participated in planning, product design, implementation reasoning, research support, debugging, documentation, and related work while GitHub provided source control, pull requests, issues, and an inspectable history of the resulting changes. The repository itself gradually became more important as a persistent source of context and project constraints.

On September 17, 2026, Codex joined that existing workflow. GitHub and source-controlled development were already part of the project, so this was not a move from informal conversation into repository-based development. The change was the addition of an implementation-oriented agent that could work directly against the repository under explicit project instructions.

The distinction will be important when later work is compared with earlier work. A difference observed after September 17 cannot automatically be attributed to Codex alone. By that point, EpochLex also had substantially more documentation, a mature architecture, QA guidance, contribution standards, a much larger repository history, explicit editorial rules, and accumulated knowledge from earlier successes and failures. Model capabilities may also have changed during the observation period, so improvements over time may reflect several variables acting simultaneously.

The retained development record establishes the product-level toolchain but does not preserve a verifiable exact ChatGPT model or configuration for the historical interactions in this observation period, nor a specific Codex model configuration for the September 17 sessions. This edition therefore identifies ChatGPT and Codex at the product/tool level and does not guess model identity from release dates or general availability.

Outside contributions require an additional distinction. EpochLex does not require all contributors to use the maintainer's AI workflow, and contributors may introduce other AI systems, development environments, or non-AI workflows. Tooling associated with a contribution should be attributed only when the development record or contributor disclosure establishes it; absence of an AI disclosure should not be interpreted as evidence that no AI was used.

6. Quantitative Evidence

The repository provides several objective measurements of project activity. During the audited development period, the repository accumulated 98 pull requests, of which 97 were merged and one was closed without merge after being superseded. The published dictionary grew from 27 entries in the first MVP to 100 entries during the initial corpus milestone and eventually to 365 published entries. At the September 18 snapshot, 183 provenance records had completed the project's initial human-reviewed research pass and 182 remained explicitly marked as pending.

These measurements demonstrate substantial project throughput, but throughput should not be confused with productivity in the experimental sense. There is no conventionally developed EpochLex control project against which elapsed time, labor, defect rates, or costs can be compared. The numbers can establish how much activity occurred and how quickly repository state changed, but they cannot independently establish how much faster AI made the work.

Token utilization is being treated similarly. The project did not maintain a complete project-specific token ledger throughout the entire observation period, but historical usage can potentially be reconstructed from development conversations and combined with measured usage where direct analytics are available. Estimated historical usage and measured usage will be reported separately, and any combined total will be expressed as a range rather than as false precision.

Model changes complicate token comparisons further. A token processed by one model or workflow is not automatically equivalent in cost, capability, context behavior, caching, or reasoning characteristics to a token processed by another. Raw token volume may therefore provide a useful measure of the scale of AI interaction, but it should not be treated by itself as a measure of productivity, cost, or intelligence.

7. What This Report Does Not Claim

The EpochLex record is substantial, but the project was not designed as a controlled comparison. This section draws a clear boundary between what the development record can support and what would require data the experiment never collected.

This report does not claim that AI made EpochLex development a particular percentage faster or saved a specific number of labor hours. The repository demonstrates high implementation and publication throughput, but no equivalent non-AI EpochLex project was created as a control. Without that comparison, an exact productivity multiplier would be speculative.

The report also does not claim that a particular percentage of EpochLex was written by AI. AI systems materially assisted with software, research support, documentation, debugging, quality assurance, and other work, but the experiment did not preserve a line-by-line record attributing every revision, prompt, tool call, or decision to a specific human or AI system. Where token usage can be reconstructed or measured, it is treated as interaction evidence rather than converted into an unsupported percentage of authorship.

The report does not claim that AI-assisted development was cheaper than conventional development. The project did not track total human labor, opportunity cost, subscription allocation, model usage cost, equivalent professional-services cost, or a comparable conventionally developed project. A later API-equivalent estimate could provide context, but it would still be a modeled comparison rather than the actual cost of creating EpochLex.

The report does not rank AI-generated code against code written by human developers. EpochLex shows that AI-assisted workflows can produce useful implementation quickly, but it also records regressions, stale behavior, incorrect assumptions, and work that required further human testing or revision. Without a matched human-only codebase, relative code quality cannot be established from this project.

EpochLex was not built autonomously, and the report does not describe it that way. Human decisions materially affected product direction, architecture, naming, editorial policy, source evaluation, interface design, testing,

acceptance, and publication. The experiment examines a human-AI collaboration rather than an autonomous software-development system.

The report also does not show that anyone could reproduce the same results simply by using the same tools. The workflow depended on substantial human evaluation, correction, prioritization, design judgment, research judgment, and decision-making. This case study documents one human-AI collaboration rather than defining a universal skill threshold for AI-assisted development.

The introduction of Codex should not be read as evidence that repository-working AI agents eliminate regressions or remove the need for human testing. One of the earliest Codex-assisted implementations received extensive automated validation and still required a follow-up correction after mobile and installed-PWA testing exposed additional pronunciation and caching behavior. Later sections examine that sequence because it shows both the strengths and limits of a more structured AI-assisted workflow.

The report does not claim that vibe coding is superior to conventional software development. In fact, one of the clearest patterns in the project is that EpochLex became increasingly dependent on familiar development disciplines as it matured. Architecture, source control, QA, issue tracking, documentation, scoped changes, validation, and human review became more explicit rather than less important.

Nor does this report claim that EpochLex's current static architecture is permanently correct. The architecture has served the project's current requirements and has influenced several implementation decisions, including the design of Word of the Day and the decision not to introduce push notifications at this stage. The repository nevertheless allows the architecture to change when future requirements provide sufficient reason to accept additional complexity.

Finally, the 365-entry corpus is not presented as evidence that EpochLex is complete. EpochLex is a Living Dictionary, and both the subject matter and the project's understanding of that subject continue to evolve. Corpus size is useful as an activity and scale measurement, but it is not a measure of completeness or editorial finality.

These boundaries are part of what makes the case study useful. AI-assisted development was changing at the same time as the product and the working method, so the report has to distinguish what the record actually shows from what can only be inferred.

8. A Note on Ambiguity

Ambiguity is not treated as a defect by default. Some of the most revealing EpochLex problems occurred when an implementation was technically reasonable but rested on an assumption that had never been defined. In those cases, the important question is not only whether the AI produced correct code, but whether the project had supplied enough information to determine what “correct” behavior meant.

For a production product, consequential ambiguity should generally be converted into durable project knowledge once it is discovered. Security, privacy, data integrity, core product behavior, architectural boundaries, materially misleading reader experiences, and difficult-to-reverse decisions should not remain unresolved merely for experimental interest. Low-risk and reversible ambiguity may sometimes be treated differently when observing how another model, tool, or workflow interprets the same situation could provide useful evidence about AI reasoning and consistency.

This creates a deliberate tension within EpochLex. The project should become more reliable as a public product, but eliminating every minor uncertainty immediately could remove potentially useful experimental observations. Later case studies therefore distinguish between ambiguity that was resolved because product responsibility required it and ambiguity that could reasonably remain open long enough to learn from the next implementation.

Part II — Reconstructing the Experiment

9. From a Term List to a Working Product

The first stage of EpochLex moved quickly enough that the repository history can make the product look more fully formed than it really was. The development conversations tell a different story. The product idea was still being discovered while the software was being built, and each working implementation forced decisions about what EpochLex should become. The earliest phase was therefore less like executing a finished specification and more like using AI-assisted development as part of product exploration.

The first working pull request, PR #1 on August 28, 2026, established several decisions that would remain important throughout the experiment. It created a responsive static dictionary with 27 terms, client-side search, A–Z navigation, category filtering, structured term data in JSON, and no framework or required build process. More importantly, the pull request described the dictionary dataset as the primary asset and the website as one interface over that structured data. That distinction established an architectural direction before the project had accumulated enough complexity to make such a decision seem consequential.

The project did not begin with a formal architecture exercise. The initial structure emerged from the immediate requirements and was reinforced as later features were added. EpochLex did not need authentication, a server-side database, user accounts, or transactional behavior, so the first implementation avoided those dependencies. Over time, that simplicity became a practical boundary against which new feature ideas could be evaluated.

The first implementation also exposed the limits of treating EpochLex as merely a searchable glossary. Once readers could browse and search the entries, questions about evidence became more difficult to ignore. A plain-language definition could explain what a term meant, but it could not answer where the term came from, whether a supposed origin was actually documented, when a phrase first appeared, or whether a source supported a definition rather than an attribution claim. These questions led directly to one of the most important changes in the project.

Later that same day, PR #5 introduced a separate provenance dataset and formalized the distinction between evidence for meaning and evidence for origin. Each published term could have a research status, origin statement, first-known-use information, history events, related terms, and claim-specific sources. Unresolved research was left unresolved rather than filled with assumptions. Within the first day, the project was already becoming both software and an editorial research system.

This mattered because the problem could not be solved simply by generating more code. The system could create fields, render sources, and enforce structure, but none of those things determined whether a historical claim was true or whether a source justified a particular wording. AI could help discover and organize evidence, while credibility, uncertainty, and attribution still required human judgment. That boundary between acceleration and editorial responsibility appeared very early in the project.

10. Rapid Expansion and the First Regression Cycle

Once the basic architecture and provenance model existed, the project expanded rapidly. Dedicated term pages, audible pronunciation, mobile navigation, mobile Browse layouts, related-term discovery, category browsing, publishing metadata, and corpus expansion were introduced through a dense sequence of pull requests on August 28. The published corpus grew from 27 entries toward an initial target of 100 while the interface simultaneously became more sophisticated.

This period shows both sides of the AI-assisted workflow. Implementation moved quickly, but several features were revised soon afterward because actual use exposed limits that were not obvious in the first pass. Rapid implementation was useful, but it was not the same thing as finished product quality. That pattern would recur throughout the experiment.

The mobile experience is an early example. PR #13 introduced accessible mobile navigation, including keyboard-operable controls, focus restoration, responsive closing behavior, and an implementation that reused existing shared navigation rather than requiring edits to every page. The solution was structurally thoughtful, but later mobile Browse work introduced interaction and layout problems that were discovered only after the changes were used in context.

PR #15 corrected regressions introduced by the preceding mobile redesign. The mobile menu needed a clearer overlay treatment, the search placeholder was too long for the available space, and the category filter was clipped by the hero container because of its overflow behavior. None of these problems meant the code was fundamentally bad. The implementation had followed one reasonable interpretation of the request while missing how the pieces behaved together on a real mobile screen.

That distinction became important later in the experiment. A generated feature could work in isolation and still be wrong at the product level. The code could execute, the control could respond, and the layout could render while the overall experience still missed the intended behavior. Testing therefore became both defect detection and a way to discover requirements that had never been stated explicitly.

The pronunciation system produced a similar lesson. Browser speech synthesis made it possible to add audible pronunciation without introducing a separate audio production system, but browser voices frequently interpreted acronyms and newly coined terms incorrectly. Later fixes added explicit pronunciation overrides for entries whose raw spelling could not be trusted to produce the intended speech output. The feature was viable because of browser capabilities, but the final behavior required project-specific knowledge that the browser and AI implementation could not infer reliably from text alone.

By the end of the initial corpus expansion, PR #23 had brought the dictionary to 100 published entries. The numerical milestone was useful because it forced the project to operate across a broader range of terminology rather than remain a demonstration with a small curated sample. It also made maintenance concerns more visible. A change that was trivial across 27 records could become materially different when it affected 100 dedicated pages, metadata records, pronunciation rules, provenance entries, and related-term relationships.

11. Quality Assurance Becomes Part of the Product

The early development cycle demonstrated that AI-assisted implementation could produce significant amounts of functional work quickly, but it also showed that the project needed repeatable methods for identifying regressions. PR #30, still within the first two days of development, established a more formal product-readiness and accessibility baseline. The change added visible focus behavior, reduced-motion handling, resilient browser-storage logic, programmatic state for controls, and documented quality checks that could be reused after later changes.

This was an important transition in the development model. Quality assurance was no longer only something performed after a visible problem appeared. It began to become part of the repository's persistent knowledge, with expectations that future changes could be checked against. The project was starting to convert individual failures into generalized safeguards.

The same pattern can be seen in the evolving documentation. Early pull requests focused mainly on creating functionality, while later work increasingly updated architecture, design, methodology, provenance, and QA documents alongside implementation. This did not eliminate documentation drift; in fact, the rapid pace of development repeatedly caused documentation to become inaccurate. What changed was the project's recognition that documentation accuracy was itself part of product quality rather than a secondary administrative task.

The resulting process was still far from a conventional large software organization. EpochLex did not acquire a formal test department, release board, or heavyweight governance structure. Instead, it developed lightweight controls that reflected the actual risks of the project: JSON integrity, parity between terms and provenance records, valid relationships, indexable dedicated pages, keyboard access, pronunciation behavior, mobile usability, cache freshness, and preservation of the static architecture.

This is one of the first places where EpochLex began to challenge a simplistic idea of vibe coding. The speed and conversational nature of the work remained, but the project was already accumulating the same kinds of constraints that mature software systems need. Faster implementation made those constraints more valuable because a wrong assumption could spread across many files or records just as quickly as a correct one.

12. The AILex to EpochLex Migration

The project's first major identity change occurred while the product was still expanding rapidly. The original short-form name, AILex, was reconsidered after broader brand and collision review. The decision was not merely cosmetic, because the name had already been incorporated into page titles, navigation, pronunciation behavior, metadata, documentation, and dedicated term pages.

PR #31 migrated the public identity from AILex to EpochLex across 123 files. The pull request changed more than two thousand lines while deliberately preserving the repository slug and existing URL structure for a later coordinated migration. This is an example of human judgment overriding implementation momentum. A substantial amount of work had already been built around the existing name, but the cost of changing it at that stage was judged to be preferable to committing more deeply to an identity that no longer seemed appropriate.

The migration also produced an immediate technical lesson. Although the current source contained the correct EpochLex wording, some mobile browsers continued displaying the old “Search AILex...” placeholder. The problem was not a failure to update the text in the repository; it was browser caching within the dynamically loaded JavaScript chain. PR #32 changed asset versioning so clients would receive the updated code.

This sequence shows why repository correctness and user-visible correctness are not the same thing. A code comparison can confirm that every intended string changed while a user's browser still shows an older version. The issue existed in the path from source code through caching and deployment to the client, not in the text replacement itself.

The rename also reinforced another recurring lesson: seemingly localized product decisions can have broad technical consequences once a project becomes interconnected. Naming affected branding, URLs, accessibility labels, metadata, pronunciation, cached assets, and documentation. The experiment increasingly demonstrated that AI could make broad changes quickly, but speed did not eliminate the need to understand the systems through which those changes propagated.

13. The Taxonomy Expands Beyond Terms

As EpochLex developed, another ambiguity emerged around the meaning of “AI terminology.” A dictionary focused only on abstract terms could explain concepts such as retrieval-augmented generation, prompt injection, or model collapse, but readers also encounter names such as ChatGPT, Claude, GPT, OpenAI, DeepSeek, and GitHub Copilot as part of the language of the AI era. The existing taxonomy did not clearly define whether those entities belonged in the same reference work.

PR #36 addressed that question by expanding EpochLex from four reader-facing categories to five and introducing explicit entry types for terms, organizations, products, model families, and individual models. The implementation retained backward compatibility for older records while allowing newly added named entities to identify their type directly. It also deliberately avoided inventing typed relationships such as “developed by,” “owned by,” or “powers,” because the existing related-term model did not support those semantics.

Subsequent pull requests tested the new structure against real examples. OpenAI, ChatGPT, GPT, Anthropic, Claude, Microsoft, GitHub, and related product and model-family entries were added in researched batches. The DeepSeek work was especially useful because the same name could refer to an organization, service, and model ecosystem, requiring an editorial decision about where separate entries provided genuine reader value.

This phase also shows that AI-assisted development was not simply filling in a predefined schema. The schema itself was being worked out through real examples. When a new class of content exposed a limitation, the project had to decide whether to expand the taxonomy, treat the item as an alias, represent it in provenance, or leave it out.

The distinction between what the data model could express and what humans might casually infer also became more important. It would have been easy to create richer semantic relationships because AI could generate them, but doing so would have introduced claims that the project had not established how to encode or maintain. The decision not to add those relationships demonstrates a form of restraint that became increasingly important as AI made implementation easier.

14. The PWA and the Boundary of the Static Architecture

PR #47 introduced the first Progressive Web App (PWA) foundation while preserving the existing static, data-driven architecture. The work added install support, offline fallback behavior, caching, and a service worker—a browser component that manages offline access and cached files—without introducing a backend, database, framework, or API. The result extended the existing site rather than creating a separate application.

The PWA later became one of the clearest examples of product intent, technical correctness, and ambiguity pulling in different directions. A site can be installable and still feel like a website with the browser frame removed. What EpochLex wanted from an installed experience became clearer only after the first version could be used and evaluated.

The mobile and installed experiences also showed the limits of simulating a device with a narrow browser window. Screen size could be approximated, but installed navigation, safe areas, cached files, browser speech behavior, and other device-specific details could still behave differently on an actual phone or tablet. Those differences later became explicit in the project's quality-assurance guidance.

At this stage, however, the ambiguity still had value. The project had not yet fully decided how different the PWA should be from the conventional website, and the first implementation created something concrete enough to evaluate. The resulting feedback became input to later requirements rather than evidence that the original implementation had simply failed.

15. From a Private Project to an Open-Source Operation

By August 30, the focus of the project began moving beyond the product itself. The repository already contained substantial documentation and contribution guidance, but it was not yet organized for someone unfamiliar with the project to find a useful way to participate. The development conversation therefore shifted toward contributor readiness and visibility within the open-source community.

PR #59 added a Code of Conduct, structured issue forms, contribution categories, a pull request template, and explicit treatment of material AI assistance. It also created bounded contribution opportunities across accessibility, code maintenance, documentation, pronunciation, provenance, metadata, and corpus research. These tasks were deliberately chosen to give contributors meaningful work without requiring them to understand the entire project before participating.

PR #64 then synchronized public and repository documentation with GitHub Discussions, the linked Project, Issues, and pull requests. The project was no longer simply using GitHub as a place to store code. GitHub had become part of the operating model through which research questions, defects, feature ideas, contribution opportunities, and implementation work could be separated and tracked.

This evolution is important because the AI experiment could otherwise be interpreted as a closed collaboration between one person and one AI system. Open-source participation changes that premise. Contributors may bring different development practices, different AI systems, conventional coding workflows, or no disclosed AI assistance at all.

For the report, this means the EpochLex development environment cannot be treated as one permanent toolchain. The maintainer workflow can be documented with reasonable confidence, but future contributions need to be described according to the evidence available for each contribution. The project should not infer tooling simply because a contribution looks AI-assisted.

16. Making the Research Pipeline Persistent

The growing dictionary also created a separate operational problem: potential terms existed across planning conversations and external working material rather than in a durable repository-controlled queue. PR #82 addressed this by creating a source-controlled corpus candidate inventory. The initial inventory contained 83 roadmap candidates and 507 broader research-pool candidates.

The inventory was intentionally not a list of future commitments. A candidate could still be rejected, combined with an existing entry, treated as an alias, or left for future observation. PR #83 reinforced this distinction by exposing the candidate pipeline publicly while stating that candidate status did not equal publication approval.

This was another sign that the project was maturing. Early development could rely heavily on recent conversation because the project was small and decisions were fresh. As the corpus, research backlog, documentation, and contribution opportunities grew, important project state needed a more durable home.

Source control increasingly became that durable home. Conversations could generate ideas and help analyze them, but a candidate that mattered to the continuing operation of EpochLex needed to survive changes in model, tool, contributor, or chat history. This later became one of the principles formalized for Codex: current repository state should take precedence over older planning material.

17. Scaling the Corpus Changes the Bottleneck

The largest quantitative expansion of EpochLex occurred on September 8. Before that expansion, the project had 122 published entries with researched provenance. The question was no longer whether additional terminology existed; the candidate inventory made clear that hundreds of plausible terms were available. The more difficult question was whether a term needed complete origin and history research before it could provide useful dictionary value.

PR #85 formalized a two-gate editorial model. The first gate asks whether an entry is sufficiently useful, defensible, and human-reviewed to publish as a dictionary entry. The second asks whether its deeper provenance record has completed an initial human-reviewed sourcing pass. A term could therefore be published while its origin, first-known-use, or historical research remained explicitly pending.

This was not a relaxation of the definition standard. The core entry still needed documented real-world usage, distinct reader value, a clear definition, example, pronunciation, classification, and matching provenance record. The change recognized that establishing what a term means and researching the full history of its usage are related but different kinds of work.

PR #86 used the new model to publish eight entries with transparent pending provenance and expand the research candidate pool. Subsequent batches continued the process, and by PR #110 the published corpus had reached 365 entries. At that moment, 122 provenance records were researched and 243 were pending.

That ratio made the new bottleneck obvious. Structured AI-assisted publishing had become fast enough that deeper provenance research could no longer keep pace. The project had not removed work; it had moved the constraint.

This is one of the strongest objective findings of the experiment because the change is visible directly in the repository state. Before the expansion, the limiting question was how to identify, define, structure, and publish enough useful terminology. After the expansion, the more pressing problem became how to review hundreds of provenance records without allowing the research layer to become permanently secondary.

PR #111 responded by creating an explicit queue for all 243 pending records. The first wave contained 61 entries selected to move EpochLex past the point where a majority of the corpus had completed initial provenance review. PRs #112 through #117 then completed that wave, resulting in 183 researched records and 182 pending.

PR #118 deliberately documented the resulting state and stopped the first consolidation phase rather than continuing automatically into another batch. That pause is important. An AI-assisted workflow made it technically possible to continue producing and researching entries, but the existence of capacity was not treated as an instruction to use it indefinitely.

The corpus phase therefore provides a useful example of a broader systems principle. Accelerating one portion of a workflow often exposes the next slowest portion rather than eliminating the workflow itself. In EpochLex, faster implementation and publication increased the importance of source verification, editorial prioritization, and research management.

18. Word of the Day and the Question of Architectural Cost

By September 15, EpochLex had a large enough corpus that features based on discovery and recurrence became more useful. A Word of the Day experience was proposed alongside the possibility of mobile notifications that could announce new terms, research updates, or daily selections. The ideas sounded related from a reader's perspective, but they had very different architectural consequences.

PR #119 implemented Word of the Day without creating a separate editorial schedule or requiring a backend process. The feature derives its selection directly from the canonical term dataset, uses Pacific Time to determine the project day, avoids recent repetition, and produces both a homepage experience and a dedicated history page. It does not require a daily GitHub Action, commit, server process, or manually maintained calendar.

The implementation is revealing because the architectural constraint did not prevent the feature from being built. Instead, the constraint shaped the solution. The project reused existing canonical data and client-side computation rather than adding infrastructure merely because a more traditional implementation might have used a scheduler or server.

Push notifications led to a different conclusion. Reliable Web Push—the browser standard behind app-like notifications—requires persistent subscription information and a trusted sender with access to private credentials. Those requirements would introduce server-side responsibilities that EpochLex did not otherwise need.

The feature was therefore deferred rather than implemented simply because AI could likely produce the necessary code. This marks an important change from the earliest stage of the experiment. The primary question had become not only whether a feature was technically possible but whether its value justified the long-term operational and architectural consequences.

19. The PWA Becomes an Intentional Product Experience

The same September 15 work also revisited the installed PWA. Earlier versions had made EpochLex installable, but subsequent design discussion clarified that an installed application should feel intentionally different from simply viewing the website without browser chrome. PR #121 introduced a standalone application shell with a compact branded header, persistent bottom navigation, dedicated Word of the Day access, and a secondary More interface.

The implementation explicitly distinguished installed mode from ordinary mobile-browser behavior. A narrow viewport alone would not trigger the application shell. This preserved the normal responsive website for browser users while allowing the installed version to behave as a separate presentation layer over the same canonical content.

That distinction had not been fully specified when the original PWA foundation was created. It became clear only after the product existed in a form that could be evaluated. In this sense, implementation was functioning as a requirements-discovery process.

The new shell also produced another cache problem almost immediately. PR #122 updated the PWA and service-worker cache versions so existing installations would actually receive the new standalone interface. Once again, the source could be correct while an existing client remained on older behavior.

Several subsequent pull requests refined the installed experience further. Navigation, filters, Browse behavior, and Word of the Day cues were adjusted based on how the PWA felt as a product rather than whether individual controls merely functioned. This series of changes provides one of the clearest examples of the role of ambiguity in the experiment.

The original request did not contain a complete specification for every characteristic of an installed EpochLex application. Instead, each working version exposed additional assumptions about what should remain website-like and what should become application-like. Some of those assumptions were then converted into durable behavior and QA expectations.

20. Introducing Codex Into an Existing Workflow

On September 17, the development environment changed again when Codex was added to the maintainer workflow. This was not the beginning of repository-based development. GitHub, pull requests, issues, source-controlled datasets, architecture documentation, QA guidance, and contribution standards were already established.

The change was more specific. ChatGPT remained the environment in which product ideas, requirements, design reasoning, research questions, and broader planning were discussed, while Codex was introduced as an implementation-oriented agent working more directly against the repository. The intention was to separate some of the product-thinking work from bounded coding tasks without removing human review from either side.

Before Codex was asked to make substantial changes, PR #127 added a root-level `AGENTS.md` file containing project-specific instructions. The document defines the current repository as the source of truth, describes the static architecture, restricts unnecessary frameworks and backends, identifies canonical datasets, explains provenance requirements, preserves the untyped relationship model, establishes design constraints, and requires scoped validation and reviewable changes.

The sequence matters because Codex was not introduced into an empty repository and expected to infer the project from code alone. It entered a project that had accumulated approximately three weeks of architecture, design, editorial, and QA knowledge. The instructions attempted to make that accumulated knowledge directly available to the implementation agent.

The first particularly revealing implementation involved Issue #60, which concerned responsive touch targets. An earlier solution existed as PR #84 from September 5, but the branch was now stale relative to current `main`. Rather than merging the old work or mechanically transplanting its diff, Codex inspected the current repository, the issue, the older proposed implementation, and the relevant current styles.

PR #128 recreated the needed behavior against current `main`, preserved a mobile typography detail that the earlier PR would have removed, and performed extensive validation. The pull request measured control sizes at multiple responsive breakpoints, checked light and dark themes, tested list and grid states, verified focus behavior, examined overflow, compared desktop geometry, and instrumented speech dispatch. PR #84 was subsequently closed without merge because the newer implementation superseded it.

This was a meaningful improvement in process structure compared with some of the earliest conversational changes. The task was bounded by an issue, the repository contained explicit constraints, an older attempt could be treated as context rather than authority, and the resulting pull request documented what had and had not been tested.

PR #128 still did not remove the need for human verification. Post-merge testing on the actual mobile and installed-PWA experience found additional pronunciation and caching problems. Agentic Workflow and Agentic Commerce

lacked the speech overrides needed for consistent pronunciation, and the service worker could continue serving an older JavaScript file from its cache even when a newer version existed.

PR #129 corrected both problems. JavaScript requests moved to a network-first strategy with controlled cache fallback, and additional pronunciation overrides were added. PR #130 then updated the architecture documentation so the written cache strategy matched the implementation.

This sequence is useful because it resists a simple success-or-failure reading. Codex produced a carefully scoped and heavily validated implementation, and real-world use still uncovered issues that required correction. The automated validation was not ineffective; it covered many relevant behaviors. The lesson is that different forms of validation catch different kinds of problems.

21. Maintenance Work Becomes More Explicit

The Codex introduction coincided with several tasks that were less visually dramatic than earlier feature development but important to the integrity of the repository. PR #131 completed the migration of legacy records to explicit ``entryType`` values. All 365 records then contained an explicit type, including 343 legacy entries updated to declare ``term`` rather than relying on runtime fallback behavior.

PR #132 followed with documentation changes so the taxonomy, roadmap, and architecture reflected the completed migration. The runtime fallback remained for backward compatibility, but it was no longer treated as the desired canonical state. This separation between migration compatibility and canonical data quality is characteristic of a project that is beginning to care about long-term maintainability rather than only current behavior.

PR #133 addressed another accumulated inconsistency. Dedicated term pages and the sitemap still contained legacy GitHub Pages URLs in places where the current custom domain should have been used. The correction changed 123 files and replaced 615 URLs while explicitly validating that no editorial content changed.

These are not the kinds of changes that make an AI-built project visually impressive, but they are important to the case study. A real product accumulates maintenance obligations even when the visible interface appears complete. Metadata, migration bridges, stale URLs, documentation, caches, schema consistency, and other forms of technical debt become part of the continuing workload.

The ability of AI tools to perform repetitive repository-wide cleanup may prove highly valuable in this phase, but the same capability raises the consequence of incorrect scope. A tool that can replace hundreds of URLs rapidly can also propagate the wrong URL hundreds of times. The later EpochLex workflow therefore increasingly paired automation with explicit validation designed to prove that repetitive changes affected only the intended fields.

22. What Changed During the First Three Weeks

The most important change between August 28 and September 17 was not a particular feature. It was the development process itself. The earliest work depended heavily on conversational intent, rapid implementation, and immediate human reaction. By the end of the observation period, the same project contained explicit architecture guidance, editorial standards, provenance rules, QA requirements, contributor workflows, source-controlled research queues, issues, pull requests, and agent-specific repository instructions.

This was not a simple transition from undisciplined development to disciplined development. Important architecture and data decisions existed from the first pull request, and human review was present from the beginning. What changed was how much project knowledge moved from implicit understanding and recent conversation into durable, inspectable project context.

The shift also changed the role of AI. Early in the experiment, AI frequently helped discover what the product might be by turning ideas into implementations quickly enough to evaluate. Later, AI increasingly operated inside

constraints created by what had already been learned. The repository became not only the place where code lived but also the mechanism through which previous mistakes, decisions, and standards could shape future AI behavior.

At the same time, the project did not attempt to remove every source of uncertainty. Some ambiguity continued to be useful because it exposed differences between technically valid implementation and intended product behavior. The increasingly important question became which ambiguities should be converted immediately into requirements and which could remain temporarily open without putting the product or its users at material risk.

The development process is therefore better described as evolving than as following a predetermined method. EpochLex did not begin with a complete AI-development framework and then test it. The framework emerged from product needs, failures, regressions, research constraints, and new tooling as the project developed.

That matters for the case studies that follow. EpochLex was not a fixed process applied to a fixed task; the product, the development method, and the AI environment evolved together. The next section focuses on the cases where that evolution is easiest to see and where the difference between implementation success, implementation failure, human ambiguity, and process maturity becomes most visible.

FIGURE 1

EpochLex Development Timeline

Major phases during the first observation period, August 28–September 17, 2026.

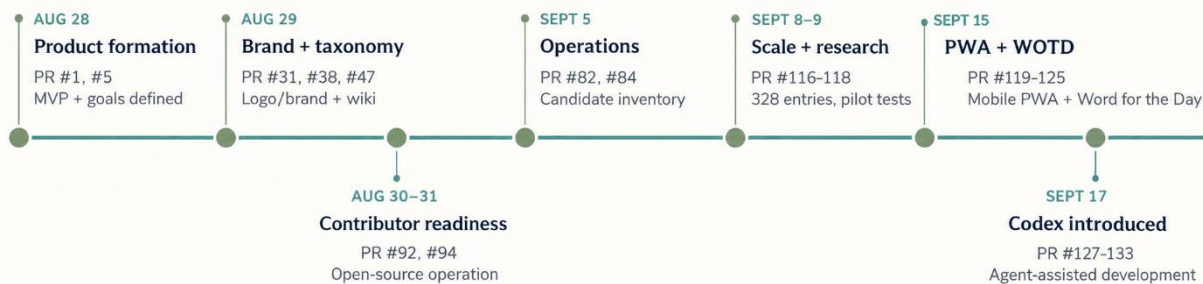


Figure 1. Major development phases during the first EpochLex observation period.

Part III — Case Studies and Findings

23. Case Study One: From 27 Terms to a Working Reference Product

Context

The first EpochLex implementation began with a relatively modest product idea: create a useful way to collect and explain terminology emerging around artificial intelligence. The initial problem was not technically complex in the conventional sense. A searchable list of terms could have been implemented quickly in many different ways, and there was no requirement for authentication, transactional data, user accounts, or server-side processing.

What made the first stage significant was not the sophistication of any individual feature but the speed at which product questions could be turned into something concrete enough to evaluate. PR #1 created a functioning 27-entry dictionary with structured JSON data, client-side search, A–Z navigation, category filtering, responsive presentation, shareable links, and contribution guidance. The initial implementation also established the principle that the structured dictionary data would be the primary asset and that the website would serve as an interface over that data.

What Happened

Once a working version existed, the discussion quickly moved beyond the original request. Search behavior raised questions about categories. Categories raised questions about how entries should be classified. Dedicated entries

raised questions about stable URLs and related terms. Definitions raised questions about sources, history, and provenance.

The early workflow was not simply AI executing a completed specification. Producing a working implementation gave the human product owner something concrete to react to and exposed requirements that had not existed in explicit form before the software was built.

Within the same initial development period, the project added provenance, dedicated term pages, pronunciation, mobile behavior, related-term discovery, publishing metadata, and a larger corpus. The number of functions and editorial responsibilities expanded much more quickly than would be suggested by the original term-list concept.

What Changed Afterward

The project gradually stopped treating rapid implementation as enough evidence that a feature was complete. As the product grew, repository documentation became more detailed, regression fixes became more common, and architectural constraints moved out of recent conversation and into durable project guidance. The first implementation remained important, but the workflow around later changes became more deliberate.

The early phase also established a pattern that would persist throughout the experiment. AI-assisted development made it inexpensive to move from an idea to an inspectable implementation, which increased the number of product decisions that could be explored in practice. At the same time, every new feature created additional surfaces that required maintenance, testing, documentation, and human judgment.

FINDING 01

The EpochLex record shows that AI materially reduced the distance between product intent and a working implementation during the early experiment. The repository demonstrates unusually rapid product expansion across several different types of work, and the conversation history shows that many product decisions were made in response to working implementations rather than before them.

EVIDENCE NOTE The evidence does not support an exact productivity multiplier. No equivalent non-AI EpochLex project was created for comparison, and the experiment did not track conventional labor hours against AI-assisted labor hours. The appropriate conclusion is therefore that AI enabled rapid iterative product exploration, not that it made development a specific percentage faster.

24. Case Study Two: Provenance Changed the Nature of the Project

Context

The original dictionary could explain what a term meant without explaining where that meaning came from. As soon as EpochLex began presenting itself as more than a casual glossary, that distinction became important. A source supporting the modern definition of a term does not necessarily establish who coined it, when it first appeared, or how its meaning evolved.

PR #5 introduced a separate provenance dataset and formalized claim-specific research. The project began distinguishing meaning, origin, first known use, historical development, related terms, and research status rather than attaching a generic list of URLs to an entry.

What Happened

This change moved a substantial portion of the project's difficulty outside software implementation. AI could generate fields, render sources, create dedicated pages, and help organize research, but those capabilities could not determine whether an attribution was justified. A source describing a term in 2025 could support its current meaning while providing no evidence that the publisher originated the term.

The project therefore adopted increasingly conservative provenance rules. Terms such as “coined by,” “introduced by,” “formalized by,” and “popularized by” were tied to different levels of evidence. First-known-use fields could remain unresolved rather than forcing an answer when the evidence was insufficient.

AI-generated research could assist with discovery and comparison, but it was explicitly excluded as a source. Human review remained responsible for determining whether a source existed, whether it supported the assigned claim, whether the wording was stronger than the evidence, and whether uncertainty should remain visible.

What Changed Afterward

Provenance became one of the distinguishing characteristics of EpochLex and eventually one of its largest operational workloads. The repository developed a dedicated PROVENANCE.md standard, separate research statuses, claim-specific source relationships, and later a structured queue for unresolved research.

The distinction between core dictionary value and deeper provenance research eventually became formal enough that PR #85 separated publication readiness from provenance completion. That decision allowed EpochLex to grow while preserving transparency about which historical questions remained unresolved.

FINDING 02

This case shows how the hardest problems in an AI-assisted product can move away from code as the product matures. The technical system for storing provenance was comparatively straightforward; deciding what the data should claim required a different kind of judgment.

EVIDENCE NOTE The case also demonstrates an important limit on AI authority. AI could help search, compare, organize, and draft research material, but the project deliberately refused to treat model output as evidence. In EpochLex, the ability to generate a plausible historical explanation was not considered equivalent to establishing one.

25. Case Study Three: Mobile Development and the Difference Between Functional and Correct

Context

The early responsive site worked across multiple viewport sizes, but mobile use quickly revealed that a technically responsive layout was not necessarily a satisfactory mobile product. Navigation, search, category filtering, card density, touch targets, and overlay behavior all required additional consideration.

PR #13 added accessible mobile navigation with keyboard support, focus restoration, and responsive state management. Subsequent work redesigned Browse for smaller screens, but PR #15 was needed almost immediately to correct regressions introduced by that redesign.

What Happened

The problems were not dramatic code failures. The mobile menu could be obscured by page content, a search placeholder did not fit comfortably, and the category selector was clipped because it sat inside a container with overflow restrictions. Each component could technically function while the overall experience was still wrong.

An automated test could confirm that a button existed and responded to a click, but it could not always establish whether the result felt coherent on an actual phone. The implementation could satisfy a narrow reading of a requirement while still missing the product owner's intended experience. That difference became increasingly important as the product matured.

The mobile work also revealed hidden assumptions about what “responsive” meant. A design that rearranged correctly at a smaller width could still be difficult to use because controls were too small, because menus were awkwardly positioned, or because the information density remained desktop-oriented.

What Changed Afterward

Mobile and responsive QA became part of the project's persistent testing guidance. Issue #52 was created specifically for real mobile and tablet testing, and Issue #60 later documented undersized touch targets with measured control dimensions. The current QA guidance distinguishes browser viewport testing from real-device testing rather than treating one as a substitute for the other.

The project also became more cautious about shared UI changes. Changes to common CSS, mobile navigation, Browse logic, pronunciation controls, or the PWA shell increasingly triggered broader regression checks instead of being evaluated only where the change was originally requested.

FINDING 03

The mobile work shows that AI-generated code can be technically valid without satisfying the intended product experience. This is not unique to AI development, but the speed of generation makes the distinction particularly important because multiple plausible interpretations can be implemented quickly.

EVIDENCE NOTE It also demonstrates that human testing is not merely a final approval step. In several instances, human use exposed requirements that had not been fully articulated before the implementation existed. Testing therefore became part of requirements discovery as well as defect detection.

26. Case Study Four: The AILex to EpochLex Rename

Context

The project originally used AILex as its short-form public name. As the product became more serious, broader identity research raised concerns about continuing to build permanent public infrastructure around that name. The project was renamed EpochLex before the earlier identity became more deeply embedded.

PR #31 migrated the public-facing brand across 123 files. Page titles, metadata, headers, accessibility labels, pronunciation behavior, documentation, and dedicated term pages were updated while the existing repository slug and URL structure were deliberately retained until a later coordinated migration.

What Happened

The decision illustrates a kind of human judgment that AI assistance did not replace. The repository already contained substantial investment in the AILex name, but the product owner decided that preserving existing work was less important than correcting the identity while the cost remained manageable.

The implementation appeared complete in the source code, but some mobile browsers still showed the old “Search AILex...” placeholder. The source was correct; the browser was loading an older cached JavaScript file. PR #32 changed asset versioning so clients would receive the updated code.

The incident showed how a seemingly simple global change can reach beyond the files directly edited. Public identity had become connected to cache state, metadata, accessibility labels, pronunciation behavior, generated pages, and browser-delivered assets.

What Changed Afterward

Caching became a recurring concern in later PWA work, and the project eventually documented service-worker and asset strategies more explicitly. The rename also influenced the way later broad repository changes were validated. Large repetitive edits increasingly included checks intended to demonstrate that only the desired fields had changed.

The branding decision additionally reinforced the principle that AI implementation should follow current project judgment rather than preserve historical decisions merely because they already existed in code. Current repository decisions gradually became the authoritative source, while earlier conversations were treated as historical context.

FINDING 04

The rename supports two findings. First, human product judgment remained capable of overriding substantial implementation inertia. Second, a successful source-code change is not necessarily equivalent to successful delivery to users.

EVIDENCE NOTE The incident also illustrates why the case study should distinguish code correctness from system correctness. The repository could contain the correct brand while a user's browser still displayed the previous identity.

27. Case Study Five: Open Source Required More Than Publishing the Code

Context

By the end of August, EpochLex already had a public repository and substantial documentation, but simply making the source visible did not make the project meaningfully contributor-ready. An unfamiliar contributor still needed to understand where to report a problem, how to suggest a term, how research corrections differed from feature requests, and what evidence or validation a pull request should include.

PR #59 introduced the missing participation infrastructure. Structured issue forms, a Code of Conduct, contribution categories, and a pull request template were added, including expectations around material AI assistance.

What Happened

The project also created bounded contribution opportunities across several domains rather than presenting outside contributors with a generic invitation to help. Accessibility review, pronunciation testing, metadata integrity, provenance research, documentation, and corpus work could each be treated as a discrete contribution.

PR #64 then clarified the relationships among GitHub Discussions, Issues, the Project, and pull requests. These tools were not interchangeable. Discussions could support open-ended conversation, Issues could define actionable work, the Project could organize work across stages, and pull requests could contain reviewable implementation.

This phase expanded the experiment beyond a single human-AI collaboration. Once outside contributions became a realistic possibility, the development environment could no longer be described as one permanent toolchain. A contributor could use a different AI system, a different editor, conventional coding practices, or no disclosed AI assistance.

What Changed Afterward

The project began treating tooling attribution more carefully. The maintainer workflow could be documented because it was known, but contributor workflows could only be described when a contributor disclosed them or when the development record established them directly. Absence of an AI disclosure could not responsibly be interpreted as proof of human-only work.

The contribution infrastructure also made project rules more durable. Editorial and technical expectations that previously existed primarily between the maintainer and AI systems had to become understandable to someone entering the repository without access to those conversations.

FINDING 05

The open-source transition shows that an AI-assisted software product needs social and operational infrastructure in addition to code. Contribution pathways, review expectations, issue boundaries, documentation, and shared terminology became necessary as soon as the project expected participation from people outside the original development loop.

EVIDENCE NOTE The evidence does not yet demonstrate that this infrastructure increases contributor volume or retention. The project had not accumulated enough external contribution history during the observation period to measure that outcome.

28. Case Study Six: Rapid Corpus Growth and the Creation of Research Debt

Context

By early September, EpochLex had moved beyond the original 100-entry milestone and maintained a large inventory of candidate terminology. The limiting problem was no longer finding enough terms. Hundreds of plausible candidates existed, but publishing each one with completed origin, first-use, history, and source research created a significant bottleneck.

PR #85 separated two editorial questions that had previously been more tightly coupled. A term could be complete enough to provide useful dictionary value while its deeper provenance research remained unfinished. The project therefore created separate gates for publication readiness and provenance completion.

What Happened

PR #86 began publishing terms under the new model with explicit researchStatus: "pending" records. The following publication batches expanded the corpus from 122 entries to 365. At that point, 122 entries had researched provenance and 243 were pending.

The numbers made the effect of increased throughput visible. Publication had accelerated, but the unresolved research workload had grown with it. The workflow had not eliminated the slower part of the process; it had allowed the faster part to run ahead.

PR #111 responded by creating a structured queue for all 243 pending provenance records. The first wave contained 61 entries selected to bring the researched portion of the dictionary above half of the total corpus. PRs #112 through #117 completed that wave, producing 183 researched entries and 182 pending entries.

What Changed Afterward

The project stopped the first consolidation wave rather than continuing indefinitely. PR #118 documented the resulting state and preserved the remaining work for later prioritization. This pause acknowledged that the existence of AI capacity did not itself determine the correct pace of publication or research.

The corpus process also became more explicit about aliases, duplicate concepts, and candidates that needed additional investigation. Increasing the number of published records was no longer treated as the only measure of progress.

FINDING 06

Rapid corpus expansion made the next bottleneck visible: source verification and provenance review. The project could publish faster than it could complete the deeper research layer.

The larger lesson is that acceleration in one part of a system does not necessarily reduce the total amount of work. It can instead relocate the constraint. In EpochLex, increased publication capacity made research prioritization and human review more important.

EVIDENCE NOTE The finding should not be interpreted as evidence that the 243 pending entries were incomplete dictionary entries. Under the established editorial model, the core definitions had met publication requirements. The backlog represented unfinished deeper provenance research, not an absence of usable dictionary content.

29. Case Study Seven: Word of the Day, Push Notifications, and Architectural Restraint

Context

The expanded corpus created opportunities for discovery features that would have been less valuable when EpochLex contained only a small number of entries. Word of the Day and mobile notifications were discussed together because both could bring readers back to the dictionary and surface changing content.

From a reader's perspective, Word of the Day and notifications were related ideas. From an architecture perspective, they were very different requests.

What Happened

PR #119 implemented Word of the Day entirely within the existing static architecture. The feature selects an entry deterministically based on the project day, avoids recent repetition, provides a dedicated page and recent-history view, and derives its content directly from the canonical dictionary data. It does not require a scheduled server process or a second editorial dataset.

Push notifications created a different problem. Reliable Web Push requires persistent subscription information and a trusted sender with access to private credentials. Those requirements would add server-side responsibilities that EpochLex did not otherwise need.

The project therefore deferred push notifications rather than treating implementation capability as sufficient justification for the architectural change. The question shifted from whether AI could build the feature to whether the feature created enough reader value to justify a persistent service and its operational responsibilities.

What Changed Afterward

The architecture documentation increasingly described static delivery as an intentional constraint rather than an accidental consequence of the original MVP. New features were expected to preserve that model where practical and to identify explicitly when a requirement would cross its boundary.

This did not make the static architecture permanent. The project remained open to architectural change where justified, but the burden shifted toward explaining why additional complexity was necessary rather than assuming that new infrastructure represented progress.

FINDING 07

The Word of the Day case demonstrates that architectural constraints can improve AI-assisted implementation by reducing the solution space. The static architecture encouraged a design that reused canonical data and avoided unnecessary infrastructure.

EVIDENCE NOTE The push-notification decision is equally important because nothing visibly “failed.” The decision not to implement something is part of the experiment. Mature AI-assisted development includes rejecting technically feasible work when its architectural cost is not yet justified.

30. Case Study Eight: A Stale Pull Request and the Introduction of Codex

Context

Issue #60 documented responsive controls that were too small for comfortable touch use. PR #84 proposed a CSS-only fix on September 5, but the pull request remained unmerged while the repository continued changing. By September 17, the implementation was no longer based on current main.

This made the issue a useful first test of the newly introduced Codex workflow. Codex had just been given repository-specific instructions through PR #127, including architecture boundaries, validation expectations, design constraints, and the requirement to work from current repository state.

What Happened

Rather than merging the old branch or applying its diff mechanically, the new implementation examined Issue #60, PR #84, current styles, and current repository history. PR #128 recreated the intended behavior against current main and preserved a mobile typography detail that the earlier implementation would have changed.

The validation was considerably more extensive than many earlier changes. Control dimensions were measured at several responsive widths, light and dark themes were checked, list and grid states were exercised, keyboard focus was verified, speech dispatch was instrumented, page overflow was tested, and desktop geometry was compared with main.

PR #84 was then closed without merge because PR #128 superseded it. The older work remained useful as historical evidence and implementation context, but it was not treated as authoritative simply because it already existed.

What Changed Afterward

The Codex handoff shows the practical value of stronger repository context. The agent could separate the underlying requirement from an outdated implementation and produce a current solution without carrying forward every detail of the stale branch.

The task also showed the practical value of explicit agent instructions. Codex did not need every EpochLex rule restated conversationally because the repository contained a durable representation of architecture, editorial constraints, validation expectations, and scope discipline.

FINDING 08

The Codex handoff shows how stronger repository context can improve AI implementation work. The agent was able to distinguish the underlying requirement from an outdated implementation and produce a current solution without reproducing every detail of the stale branch.

EVIDENCE NOTE The evidence is not sufficient to claim that Codex was inherently better than the previous workflow. By September 17, the repository contained much richer project context, the task itself was tightly bounded, and the development process had accumulated several weeks of learning. Model capability, tooling, repository maturity, and task structure changed simultaneously.

31. Case Study Nine: Strong Automated Validation Did Not Eliminate Human Testing

Context

PR #128 is one of the most thoroughly validated implementations in the observed period. The pull request documented responsive measurements, keyboard behavior, visual checks, speech-event testing, boundary conditions, theme behavior, and comparisons against current main. If automated validation alone were sufficient to establish product correctness, the work should have been close to complete.

What Happened

Post-merge testing on the actual mobile and installed-PWA experience revealed two additional problems. Agentic Workflow and Agentic Commerce did not contain the explicit pronunciation overrides necessary to produce consistent speech, and the service worker could continue serving an older JavaScript version because of the interaction between cache lookup and the existing stale-while-revalidate strategy.

PR #129 corrected both issues. Additional speech overrides were added, the service-worker cache version changed, JavaScript requests became network-first when online, and fallback behavior was limited to the appropriate current caches. PR #130 then brought the architecture documentation into alignment with the implemented strategy.

The important point is not that PR #128's testing was useless because another defect appeared. It successfully verified many relevant behaviors. The later problems simply fell outside the conditions those tests covered.

What Changed Afterward

The project became more explicit about reporting what a validation process had not tested. Current QA guidance distinguishes simulated browser testing from physical-device testing, event dispatch from audible pronunciation quality, and repository correctness from installed-PWA cache behavior.

The workflow also began treating human review as one layer in a validation system rather than as a binary alternative to automation. Automated testing, code inspection, browser instrumentation, real-device checks, and human product judgment each detect different classes of problems.

FINDING 09

The PR #128/#129 sequence shows that structured AI-assisted QA can improve confidence without eliminating human validation. The stronger later testing process caught more problems before merge, but it did not make the system omniscient.

EVIDENCE NOTE The case also cautions against measuring development quality only by whether follow-up fixes occur. A mature validation process may expose more defects precisely because it is examining the product more thoroughly.

32. Case Study Ten: Ambiguity as Part of the Experiment

Context

A discussion about AI-assisted development raised a more subtle issue than code quality. An AI system can produce code that is technically sound, internally consistent, and reasonable given the available information while still delivering the wrong product behavior. In such a case, the central failure may not be an implementation error. The missing element may be a requirement that the human product owner never realized needed to be stated.

This distinction is especially important for EpochLex because the project serves two purposes at once. It is a real public product that should become more reliable over time, but it is also an experiment intended to reveal how AI systems interpret requirements, context, and assumptions.

What Happened

The PWA and mobile experience provides a useful example. The early PWA was installable and functional, but that did not answer what an installed EpochLex experience should feel like. A web application can run without browser chrome and still behave largely like the website from which it came.

As the PWA was used and reviewed, several unstated product expectations became visible. The installed application should have a more intentional navigation model, but normal mobile-browser use should retain the website interface. Responsive viewport width should not accidentally trigger installed-app behavior. Important destinations should remain easily reachable, and the installed shell should feel consistent with EpochLex rather than like a generic standalone website.

Those requirements were not all present in the first PWA request. They emerged because a technically functioning implementation created something concrete enough for the product owner to evaluate.

PR #121 eventually encoded a much clearer distinction between installed mode and normal browser mode. Later refinements continued to adjust the PWA based on intended experience rather than minimal technical functionality.

When Ambiguity Becomes a Liability

A production system cannot leave every uncertainty open merely because observing AI behavior is interesting. Ambiguity that affects security, privacy, credentials, canonical data, source integrity, materially misleading reader claims, destructive behavior, architectural commitments, or fundamental product behavior should generally be resolved as soon as it is recognized.

These areas have consequences that exceed the experimental value of seeing how another model might interpret the same uncertainty. A silent assumption involving credentials or data corruption cannot reasonably be preserved simply to compare future model behavior.

The same principle applies to ambiguities that would repeatedly create substantial rework. Once the project has established that a particular behavior matters to the product, failing to document that behavior adds little experimental value and makes future implementations unnecessarily inconsistent.

When Ambiguity May Remain Useful

Low-risk ambiguity can have a different value. A minor interaction preference, reversible presentation choice, or non-destructive workflow assumption may remain unresolved temporarily if observing another implementation could reveal something useful about AI reasoning. The project may want to know whether the same model makes the same assumption twice, whether another model interprets the requirement differently, whether richer repository context causes the ambiguity to disappear without a new instruction, or whether an AI system recognizes that the decision is missing and surfaces it rather than silently choosing. In those cases, immediately converting every observed preference into a permanent rule could make the software more deterministic while making the experiment less informative.

What Changed Afterward

The ambiguity discussion suggests that EpochLex should begin recording a more detailed classification when an AI-assisted implementation produces an unexpected result. The cause should not automatically be labeled as “AI error.”

A future evidence register could distinguish implementation error, missing repository context, unstated human requirement, conflicting instructions, outdated project knowledge, and intentionally unresolved ambiguity. That classification would tell us more than a simple bug count because it separates execution failures from missing context and missing intent.

The project should also consider whether an AI system identifies missing decisions on its own. A development agent that distinguishes a harmless implementation detail from an unresolved architectural decision may be more useful than one that simply produces plausible code in every situation.

FINDING 10

This case supports one of the broader conclusions of the EpochLex experiment: the failure is not always that the AI wrote bad code. Sometimes the implementation exposes a requirement that had previously existed only as an unstated human assumption.

EVIDENCE NOTE For a real product, consequential ambiguity should become durable project knowledge once discovered. For an AI-development experiment, low-risk ambiguity may sometimes remain open long enough to observe whether the same assumption recurs or whether another AI system handles it differently.

The practical boundary is therefore not whether ambiguity exists but what happens if it remains unresolved. Product responsibility should take precedence where the consequence is material, while reversible low-risk uncertainty may remain useful experimental evidence.

FIGURE 2

Ambiguity decision boundary

EpochLex treats ambiguity according to the consequence of leaving it unresolved, not simply whether ambiguity exists.

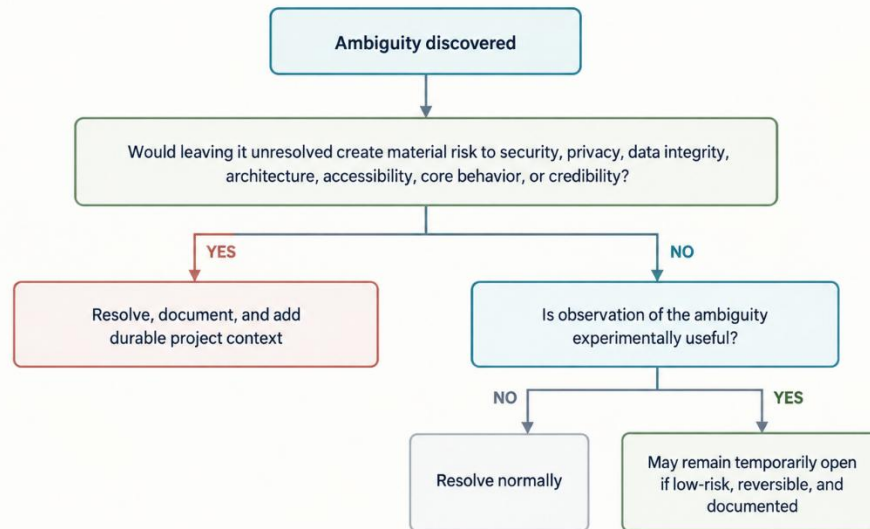


Figure 2. Practical boundary for resolving or temporarily preserving ambiguity.

33. Cross-Case Findings

The individual cases differ substantially, but several patterns appear repeatedly across the development record. The first is that AI-assisted development increased the number of ideas that could be converted into something testable. This was visible in the initial MVP, the rapid interface expansion, the PWA, Word of the Day, taxonomy changes, and later maintenance work. The ability to generate implementation quickly made experimentation less expensive and allowed product decisions to be informed by actual behavior rather than speculation alone.

The second pattern is that faster implementation increased rather than reduced the importance of review. Mobile regressions, pronunciation errors, caching behavior, provenance uncertainty, stale documentation, and schema cleanup all demonstrate that implementation speed can propagate incorrect assumptions as efficiently as correct ones. Human review, automated validation, architecture constraints, and durable documentation therefore became more important as the project expanded.

A third pattern is that the location of the bottleneck changed over time. Early in the project, generating a useful product and sufficient corpus content were primary challenges. Once implementation and publication accelerated, provenance research, source verification, maintenance, and editorial prioritization became comparatively more significant.

The fourth pattern is that project context became a form of infrastructure. Architecture documentation, QA guidance, provenance standards, contribution rules, issues, source-controlled candidate queues, and AGENTS.md increasingly influenced what future AI systems could reasonably infer about the project. The repository was no longer only a storage location for code; it became part of the mechanism by which previous human judgments constrained future AI work.

The fifth pattern is that ambiguity frequently surfaced through implementation rather than before it. Some of these ambiguities represented defects in the development process and needed to be closed. Others provided useful information about assumptions that neither the human nor the AI had previously been required to articulate.

Taken together, the most consequential effect of AI in EpochLex was not simply code generation. AI changed the cost and pace of moving between intention, implementation, observation, and revision. The rest of the development system then had to adapt to that faster cycle.

34. A Developing Interpretation of “Vibe Coding”

The term “vibe coding” is useful in describing the conversational origin of much of the EpochLex implementation, but the repository history shows that the term becomes less precise as the project matures. Early changes often began with natural-language descriptions of desired behavior followed by generated implementation and human feedback. That pattern remained present throughout the experiment.

What changed was the amount of formal context surrounding the conversation. Requirements increasingly referenced current architecture, open issues, documented design decisions, provenance standards, QA guidance, and established data structures. Later AI implementation was therefore conversational without being context-free.

Informal prompting and disciplined engineering are sometimes treated as opposites, but EpochLex does not support that split. The project continued to use conversational AI while adding increasingly conventional engineering controls around it.

A more useful description of the mature workflow is that natural-language direction became an interface to a progressively more structured development system. The human still described problems conversationally, but the AI was expected to work within durable constraints rather than reconstruct the project from each prompt.

35. The Role of Human Judgment

The repository and conversation record also make it difficult to characterize the human role as simple approval. Human decisions changed the name of the product, rejected visual directions, altered the taxonomy, established provenance thresholds, separated publication from research completion, deferred push notifications, refined the PWA, identified mobile problems, and determined when development should stop for consolidation.

These decisions frequently concerned questions for which there was no objectively correct implementation. The architecture could support several alternatives, and the code could be made to function under more than one interpretation. The human role therefore included deciding what kind of product EpochLex should be, not simply determining whether generated code executed correctly.

This is particularly evident in editorial work. An AI system can produce a plausible definition or historical narrative, but EpochLex requires a human decision about whether the evidence justifies publication. The human role is therefore intertwined with accountability for what the product asserts publicly.

The record supports describing AI as a substantial development collaborator without describing the project as autonomously directed. Human agency remained visible at nearly every important point where implementation possibility had to become a product decision.

36. The Emerging Development Model

By the end of the initial observation period, the EpochLex workflow had developed into a layered system. ChatGPT remained useful for product exploration, research, requirements, design reasoning, and broader planning. Codex had been added for more bounded repository implementation work. GitHub provided the persistent record through issues, branches, pull requests, documentation, and current main.

These tools did not replace one another. Their roles overlapped, but they increasingly served different parts of the workflow. The human project owner remained responsible for directing the work, evaluating tradeoffs, testing actual behavior, and deciding what should merge or publish.

Outside contributors introduce an additional layer of variability. Future EpochLex work may involve other AI systems, conventional development environments, or combinations that do not resemble the maintainer workflow. The case study therefore treats tools as part of the observed environment rather than as permanent requirements of the project.

This model was still new at the end of the observation period. Codex entered too late to determine whether the new division of labor reduces rework, improves quality-assurance coverage, changes token usage, or alters the kinds of errors that appear. Those questions need to be tracked in the next phase rather than inferred from a handful of early tasks.

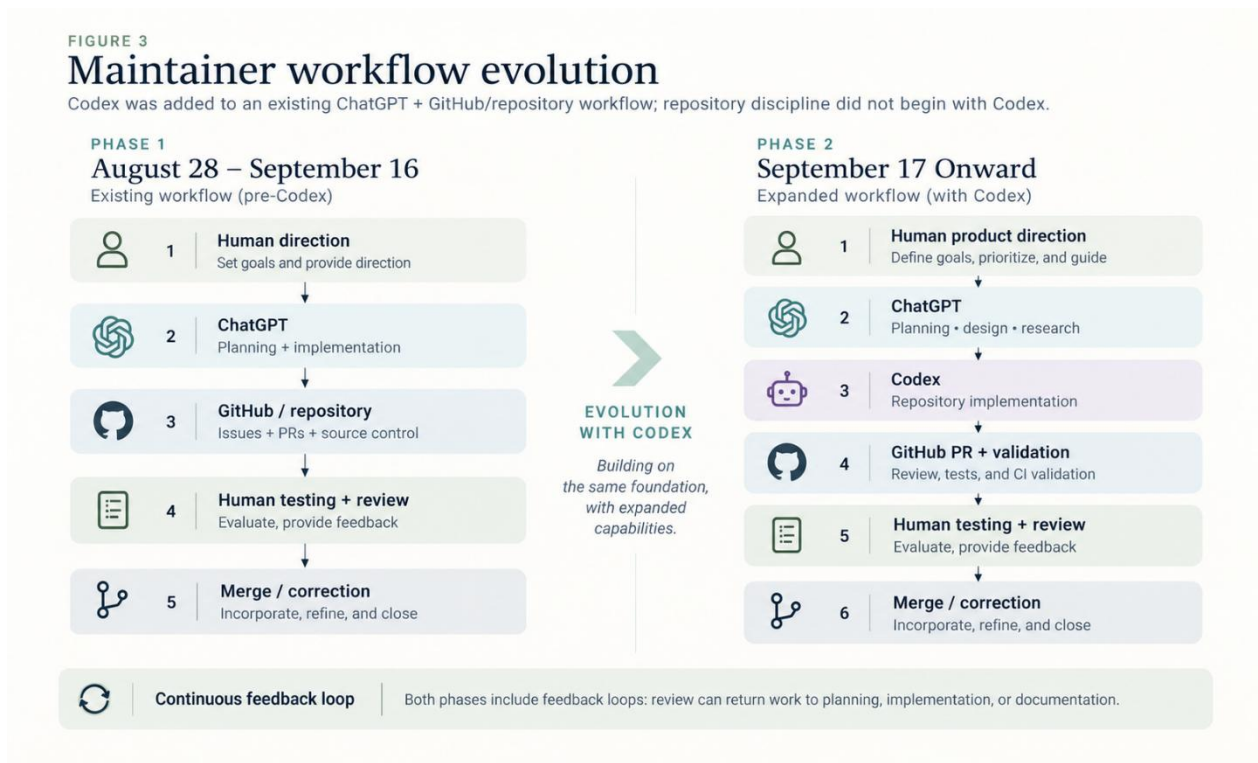


Figure 3. The maintainer workflow before and after Codex was added on September 17.

37. Interim Finding

The first phase of the EpochLex experiment does not support a conclusion that AI removes the need for software-development discipline. The evidence points in the opposite direction. As AI made it easier to generate software, data, and documentation, the project increasingly formalized architecture, validation, editorial standards, source control, contribution processes, and human review.

At the same time, the experiment does not support the conclusion that AI merely automates conventional coding tasks. The development conversations show AI participating in product exploration, research organization, requirements discovery, debugging, documentation, and analysis across a much broader portion of the lifecycle.

The clearest reading at this stage is that AI changes the speed and cost of iteration more clearly than it changes the need for judgment. EpochLex could move from idea to implementation quickly enough that working software itself became part of the thinking process. The value of that acceleration depended increasingly on whether the surrounding development process could distinguish useful output from plausible but incomplete output. That distinction provides the foundation for the next part of the report, which should examine the evidence quantitatively and consider what can reasonably be measured about throughput, token use, model changes, rework, and human intervention without converting incomplete data into false precision.

Part IV — Quantitative Analysis and Experimental Measures

38. Why Quantitative Measures Matter

The EpochLex repository contains enough activity to support quantitative analysis, but the available measurements are not all equally meaningful. Pull requests, published entries, provenance states, issues, and repository changes can be counted directly. Development time, productivity improvement, token consumption, human effort, and comparative cost are more difficult because those values were not tracked consistently from the beginning of the experiment.

This section therefore separates direct measurements from reconstructed estimates and prospective measurements. The purpose is not to produce as many numbers as possible, but to identify which numbers actually describe the experiment and which would create an appearance of precision that the evidence cannot support.

The distinction is particularly important because EpochLex developed rapidly. High activity can demonstrate throughput, but throughput alone does not establish efficiency. A project can produce many pull requests because work is proceeding quickly, because work has been divided into unusually small units, because implementation requires substantial rework, or because all three are occurring simultaneously.

For that reason, the quantitative record is most useful when interpreted alongside the qualitative case studies. The numbers describe what occurred; the case studies help explain why.

39. Repository Activity During the Observation Period

The audited repository contains 98 pull requests associated with the development period examined in this report. Of those, 97 were merged and one was closed without merge after being superseded by a later implementation. The unmerged pull request, PR #84, remains useful experimental evidence because it demonstrates how a previously valid implementation could become stale while the repository continued evolving.

The issue record contains 30 identified GitHub issues within the audited set. At the September 18 repository snapshot, 25 were closed and five remained open. The remaining issues were concentrated largely in areas requiring human or semi-manual review, including keyboard accessibility, pronunciation quality, contributor-documentation integrity, status consistency, and provenance-source quality.

This distribution is notable because the unresolved work was not dominated by features that simply needed additional code. Several remaining tasks required judgment, real-device or human testing, source evaluation, or consistency review. That pattern is consistent with the broader observation that the project bottleneck gradually moved away from basic implementation.

Repository Snapshot

Measure	September 18, 2026 snapshot
Pull requests in audited history	98
Merged pull requests	97
Closed without merge	1
GitHub issues in audited set	30
Closed issues	25
Open issues	5

Measure	September 18, 2026 snapshot
Initial published corpus	27 entries
First corpus milestone	100 entries
Current published corpus	365 entries
Researched provenance records	183
Pending provenance records	182
Provenance records completed in Wave 1	61
Reader-facing editorial categories	5
Supported explicit entry types	5
Legacy records migrated to explicit entryType	343
Legacy public URL replacements in PR #133	615
Files affected by PR #133	123

These numbers are descriptive, not interchangeable measures of progress. Corpus counts describe product content; pull requests describe units of repository change whose sizes vary widely; provenance counts describe research state; and issue counts depend partly on how the project chooses to divide work.

The table is useful because it establishes the scale of the project at the reporting cutoff without turning unlike measurements into a single score.

40. Development Was Highly Concentrated in Bursts

Repository activity was not distributed evenly across the observation period. The project experienced several concentrated development bursts corresponding to major changes in focus.

Forty-one of the 98 pull requests were opened on August 28 and 29, when the core product, initial corpus, provenance system, mobile behavior, publishing foundation, QA baseline, branding, taxonomy, and early PWA work were being established. September 8 produced another 33 pull requests, dominated by corpus expansion and provenance operations. September 15 produced eight pull requests associated largely with Word of the Day and the installed PWA experience, while September 17 produced seven pull requests around Codex introduction, accessibility cleanup, pronunciation, cache strategy, schema migration, and publishing integrity.

The concentration is useful because it reinforces the phase structure reconstructed earlier in the report. EpochLex did not progress through a steady stream of similarly sized changes. Development accelerated around specific product questions and then slowed or shifted when a different kind of work became limiting.

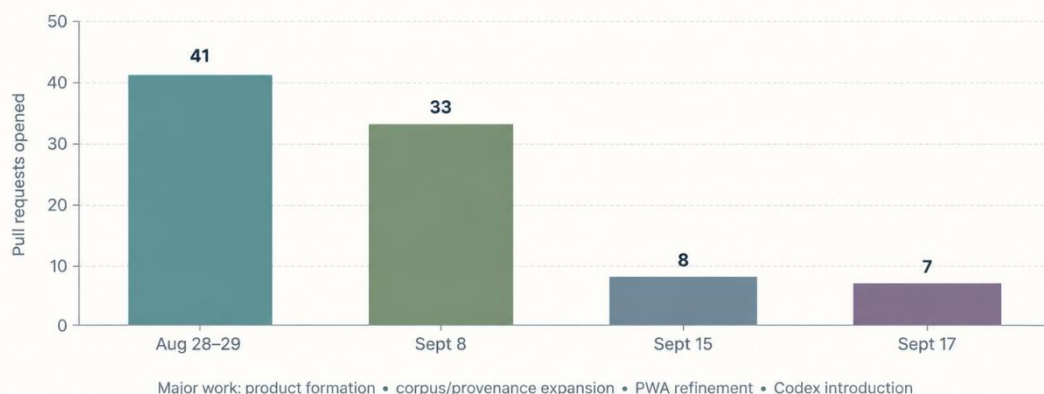
Major Activity Bursts

Period	Pull requests opened	Dominant work
August 28–29	41	Product formation, provenance, 100-entry corpus, mobile, QA, branding, taxonomy, PWA
September 8	33	Corpus expansion, research pipeline, provenance consolidation
September 15	8	Word of the Day, standalone PWA refinement
September 17	7	Codex introduction, touch targets, pronunciation/cache corrections, maintenance

FIGURE 4

Development activity concentrated in bursts

Pull-request counts show periods of concentrated repository change; they are not a measure of labor hours or productivity.



KEY TAKEAWAY

Development activity occurred in distinct bursts, with the highest number of pull requests opened on August 28–29. These peaks reflect focused phases of work rather than a continuous, linear pace.

Figure 4. Pull requests opened during the four largest activity bursts. Counts indicate repository activity, not labor hours or productivity.

These bursts should not be interpreted as equivalent to labor hours. A pull request containing a three-line cache version change and a pull request affecting more than one hundred files both count as one pull request. The activity pattern is nevertheless useful for locating periods of unusually high project change and comparing those periods with the qualitative record.

41. Corpus Growth

The published corpus is one of the clearest measures of project scale because the underlying unit is relatively stable: a published EpochLex entry.

Before the September 8 expansion, EpochLex contained 122 published entries. After the publication batches completed that day, the corpus contained 365 entries. The increase therefore represented 243 additional published entries within that expansion phase.

This is strong evidence of high publication throughput. It is not, by itself, evidence of equivalent growth in completed research because the publication model had changed shortly before the expansion.

That distinction is essential. PR #85 separated publishable dictionary completeness from deeper provenance completion. The subsequent 243-entry increase therefore measured the project's ability to review and publish usable dictionary entries under the new two-gate model, not the completion of full provenance research for all 243 entries.

42. Provenance as a Quantifiable Bottleneck

The September corpus expansion created one of the clearest quantitative examples of a shifting bottleneck. At 365 entries, 122 provenance records were marked researched and 243 were pending. Approximately two-thirds of the corpus therefore still required an initial dedicated provenance review, even though the underlying dictionary entries had met the separate standard for publication.

PR #111 converted that backlog into an operational queue. Wave 1 contained 61 records selected in part to move the corpus past the point where a majority of entries had completed the initial provenance review.

PRs #112 through #117 completed all 61 records. The resulting state was 183 researched and 182 pending.

The importance of these figures lies in the transition they reveal. Publication capacity increased and immediately created a larger research workload. After the first consolidation wave, the project deliberately paused instead of treating available AI capacity as a reason to begin the next batch automatically.

The numbers make the shift visible: faster production did not remove the constraint; it moved it. In this case, the constraint shifted from identifying and publishing useful terminology toward verifying history, attribution, and source support.

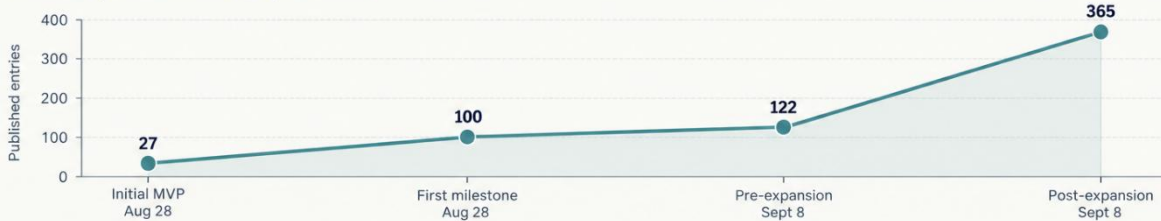
FIGURE 5

Corpus growth and provenance consolidation

Published entries grew rapidly; deeper provenance research became the next visible constraint.

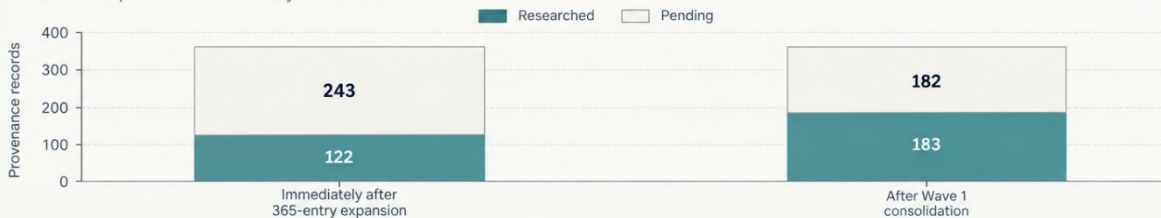
PUBLISHED ENTRIES OVER TIME

Cumulative published entries at key milestones.



PROVENANCE RECORDS BEFORE AND AFTER CONSOLIDATION

Breakdown of provenance records by research status.



KEY TAKEAWAY

Published entries increased from 27 to 365, while provenance research became a larger focus, with a more balanced mix of researched and pending records after consolidation.

Figure 5. Corpus growth and the shift from publication throughput to provenance-research pressure.

43. Pull Requests as a Measure of Iteration, Not Productivity

The 98 pull requests provide a detailed development history, but they are not a direct productivity metric. Their scope varied widely: PR #122 changed only a handful of cache-version values, while PR #31 migrated the product identity across 123 files and PR #133 performed 615 controlled URL replacements across another 123 files.

Their more useful value is as a record of iteration. Pull requests make it possible to observe the sequence between feature implementation, discovered problem, correction, documentation update, and maintenance work.

Several sequences are especially informative:

- mobile redesign followed by PR #15 regression fixes;
- brand migration followed by PR #32 cache correction;
- standalone PWA implementation followed by PR #122 cache correction;
- Codex touch-target implementation followed by PR #129 pronunciation and service-worker corrections;
- cache-strategy implementation followed by PR #130 documentation alignment;

- schema migration in PR #131 followed by documentation synchronization in PR #132.

These sequences provide a basis for studying rework, but they should not automatically be counted as failures. A follow-up pull request may reflect a defect, an improvement discovered through use, documentation catching up with implementation, or a deliberately staged change. Any meaningful rework measure therefore needs classification rather than a simple count.

A retrospective primary-intent classification of all 98 audited pull requests adds another perspective on the workload. Fifty are primarily editorial/data work, compared with 10 reader-facing product features; the remainder are corrective fixes (8), documentation (8), iterative refinement (7), maintenance/migration (5), QA/accessibility (4), governance/community (3), and architecture/infrastructure (3). Because many pull requests touch more than one concern, Appendix B treats this as an interpretive coding of primary intent rather than a native repository attribute or a measure of effort.

44. Rework and Follow-Up Work

The repository contains enough information to begin classifying rework retrospectively, although a precise rework rate would require a more systematic review of all 98 pull requests. For the purposes of this case study, follow-up work should be divided into at least four categories.

Corrective rework addresses behavior that did not meet the intended requirement. PR #15 correcting clipped mobile interfaces is a clear example.

Deployment or state correction addresses the difference between correct source and incorrect delivered behavior. PR #32 and PR #122 fall largely into this category because stale cached assets prevented users from receiving already-correct source behavior.

Iterative product refinement occurs when an implementation works as specified but use reveals a better or more appropriate experience. Several of the standalone PWA pull requests fit this category.

Documentation or maintenance synchronization aligns project knowledge with a change that is already technically correct. PR #130 and PR #132 are examples.

Those distinctions matter because combining all four categories into one “AI rework rate” would hide what actually happened. Some follow-up work represents bugs; other work represents normal product learning, deployment behavior, or deliberate staging.

A future quantitative analysis could classify every pull request according to feature development, corrective rework, iterative refinement, editorial/data work, documentation, architecture, QA, governance, or infrastructure. The current audit already provides enough evidence to justify such a classification, but the resulting percentages should be calculated only after each pull request has been reviewed consistently.

45. Human-Detected Versus Automated Findings

Another useful future measurement concerns how problems are discovered. The current record includes defects found through direct use, automated validation, repository inspection, source research, and structured review. PR #128 is especially valuable because it documents extensive automated and instrumented testing while also recording the limitations of those tests. PR #129 then documents issues found through subsequent mobile and installed-PWA verification.

This creates the beginning of a measurable distinction between problems detected before merge and problems detected through post-merge use. The current sample is too small to establish a meaningful defect rate, but the classification could be applied prospectively from the Codex phase onward.

For future work, each discovered issue could record its discovery source:

- automated repository validation;
- browser automation;
- AI code/repository review;
- human desktop testing;
- human mobile-device testing;
- installed-PWA testing;
- accessibility review;
- editorial review;
- provenance/source verification;
- external contributor or reader report.

Over time, this would allow EpochLex to examine which classes of problems are routinely missed by particular validation methods. That would be more useful than treating “testing” as one undifferentiated activity.

46. The Toolchain Timeline

The development environment changed during the experiment, so the toolchain is part of the record. For most of the observation period, the known maintainer workflow was ChatGPT combined with GitHub and repository-based development. ChatGPT supported product thinking, implementation, research, debugging, documentation, design analysis, and related work, while GitHub provided the persistent code and work-management record.

On September 17, Codex was added to that existing system. The maintainer workflow therefore became ChatGPT + Codex + GitHub/repository development rather than replacing ChatGPT with Codex.

Known Toolchain Phases

Period	Maintainer environment	Status
August 28–September 16	ChatGPT + GitHub/repository workflow	Established from project history
September 17 onward	ChatGPT + Codex + GitHub/repository workflow	Established
Contributor work	Tooling varies by contributor	Record only when known or disclosed

The underlying ChatGPT model should be recorded separately from the product-level workflow where it can be verified. The current report should not infer model versions solely from dates or assume that all conversations used the same model configuration.

This matters because later differences in performance may have several explanations. The repository contained more context, the process was more mature, Codex had been added, tasks may have been better bounded, and the underlying models may have changed. The first observation period does not isolate those factors.

47. Model Changes Are Part of the Experiment

AI models should not be treated as interchangeable instruments throughout the observation period. Differences in reasoning, context management, code generation, tool use, and output behavior may affect development results even when the human workflow appears similar.

The model-reconstruction pass for this edition did not identify a reliable historical record naming the exact ChatGPT model or configuration used before September 17, or the exact Codex configuration used in the first repository sessions. Product-level use of ChatGPT and the September 17 introduction of Codex are established; model-level attribution remains unspecified.

That gap is preserved rather than filled from general release dates. A model being publicly available on a particular date does not establish that it powered a particular EpochLex interaction, and later differences in output cannot be assigned to an unverified model change.

The model timeline for this edition is therefore limited to the verified product/tool level shown below.

Date range	Product/tool	Model	Evidence quality
Early EpochLex development	ChatGPT	Exact model/configuration not recoverable from retained evidence	Product-level workflow verified
Later pre-Codex development	ChatGPT	Exact model/configuration not recoverable from retained evidence	Product-level workflow verified
September 17 onward	ChatGPT + Codex	Exact historical model/configuration not recoverable from retained evidence	Toolchain transition verified

Model capability remains a plausible influence on the results, but it is unmeasured in this first observation period. The report therefore does not attribute later improvements to a model change in isolation, especially because repository context, task definition, tooling, and human experience were changing at the same time.

48. Token Utilization

Token consumption could help show the scale of interaction between the human developer and AI systems, but the evidence set for this edition does not support a defensible lifetime total. EpochLex did not begin with project-specific token accounting, and the retained record does not contain a complete transcript set, repeated-context history, or project-attributable Codex/API analytics for the full observation period.

A valid reconstruction would require complete EpochLex conversation transcripts, a documented tokenizer or approximation, and a method for separating unique visible text from context processed repeatedly across turns. Direct Codex or API analytics would need to remain separate from reconstructed ChatGPT history so measured and modeled values are not blended together.

Because those records are incomplete, this edition reports token usage as unavailable rather than substituting a speculative estimate. The method remains documented so a later edition can add the measurement if sufficiently complete source data becomes available.

The evidence status for this edition is summarized below.

Usage category	Value	Evidence
Historical visible ChatGPT conversation tokens	Not reported	Complete project transcript set not available in this edition
Estimated historical model input/output range	Not reported	Insufficient basis without complete transcript/context reconstruction
Measured Codex usage	Not available in evidence set	No project-attributable analytics supplied for this edition

Usage category	Value	Evidence
Measured API usage	Not established	No project-attributable API usage record in this edition
Total experimental range	Not reported	Would combine incomplete measured and reconstructed components

No exact or estimated lifetime token figure is reported in this edition. This is an evidentiary limit of the first observation period, not a zero-use result.

49. Token Counts Are Not Productivity Scores

Even if a token estimate becomes available later, it will need careful interpretation. Different models use tokens differently, and cached input, uncached input, output, tool context, and context-window behavior all affect how much processing a task requires. A million tokens in one model or workflow is not automatically equivalent in capability or cost to a million tokens in another.

The same limitation applies to financial comparisons. Much of the EpochLex experiment occurred through subscription products rather than through a dedicated API billing account. Converting all reconstructed usage into current API pricing could provide an illustrative equivalent, but it would not represent the actual amount paid to develop EpochLex.

Token volume is best understood as a measure of interaction scale. It may become useful for comparing later phases of the experiment if model, tool, and task categories are also recorded. It should not be treated as a standalone efficiency metric.

50. Human Effort Is the Largest Missing Quantitative Measure

The experiment currently lacks a reliable record of human time spent on EpochLex. Conversation timestamps and pull-request timing can show when activity occurred, but they cannot establish continuous working time. Long gaps may represent unrelated work, sleep, research outside ChatGPT, device testing, or simply leaving a conversation open.

This prevents the report from making a credible statement such as “EpochLex required X human hours.” Any such estimate would currently depend too heavily on assumptions.

The missing value is significant because human effort is necessary for several comparisons that might otherwise seem attractive. Without it, the report cannot calculate reliable productivity per hour, equivalent professional-services cost, or the ratio between AI processing and human review time.

Prospective collection could solve this problem without requiring invasive time tracking. Major future tasks could record approximate human involvement bands such as less than 15 minutes, 15–30 minutes, 30–60 minutes, one to two hours, or more than two hours. This would provide useful directional data while recognizing that exact minute-by-minute accounting may be disproportionate to the experiment.

51. Measuring the Next Phase Beginning With Codex

The introduction of Codex provides a natural point for more deliberate measurement. Earlier phases have to be reconstructed from repository and conversation history; future work can be recorded as it happens.

A lightweight task record could include:

- date;
- originating issue or requirement;
- task category;

- AI tool or tools used;
- model, when verifiable;
- whether requirements were judged clear before implementation;
- whether ambiguity was identified by the AI;
- approximate token usage where available;
- validation performed before review;
- human review findings;
- whether the implementation was accepted without change;
- whether a follow-up corrective PR was required;
- whether a later regression was traced to the change;
- approximate human review effort;
- final disposition.

The purpose would not be to create bureaucratic overhead around every minor edit. The record should be lightweight enough that collecting the data does not substantially alter the development behavior being studied.

Over time, these measures could begin answering questions that the initial retrospective cannot. The experiment could compare first-pass acceptance, human intervention, corrective rework, ambiguity detection, validation coverage, and token consumption across different toolchains or model generations.

52. Tracking Ambiguity as Data

The ambiguity case study also suggests a new class of quantitative evidence.

When an implementation produces an unexpected result, the cause could be classified as:

1. implementation error;
2. missing repository context;
3. unstated human requirement;
4. conflicting requirements;
5. outdated project information;
6. tool or environment behavior;
7. deliberately unresolved ambiguity;
8. not yet determined.

This classification would prevent every unexpected result from being counted as an AI coding failure. It would also make it possible to see whether some models or workflows are better at identifying ambiguity before implementation rather than silently choosing an interpretation.

Over time, the project could ask how often a model surfaced a missing decision, how often human review revealed an unstated requirement, and how often the same unresolved assumption reappeared across separate implementations.

That would add something conventional defect counts do not capture: the interaction between incomplete human intent and AI inference.

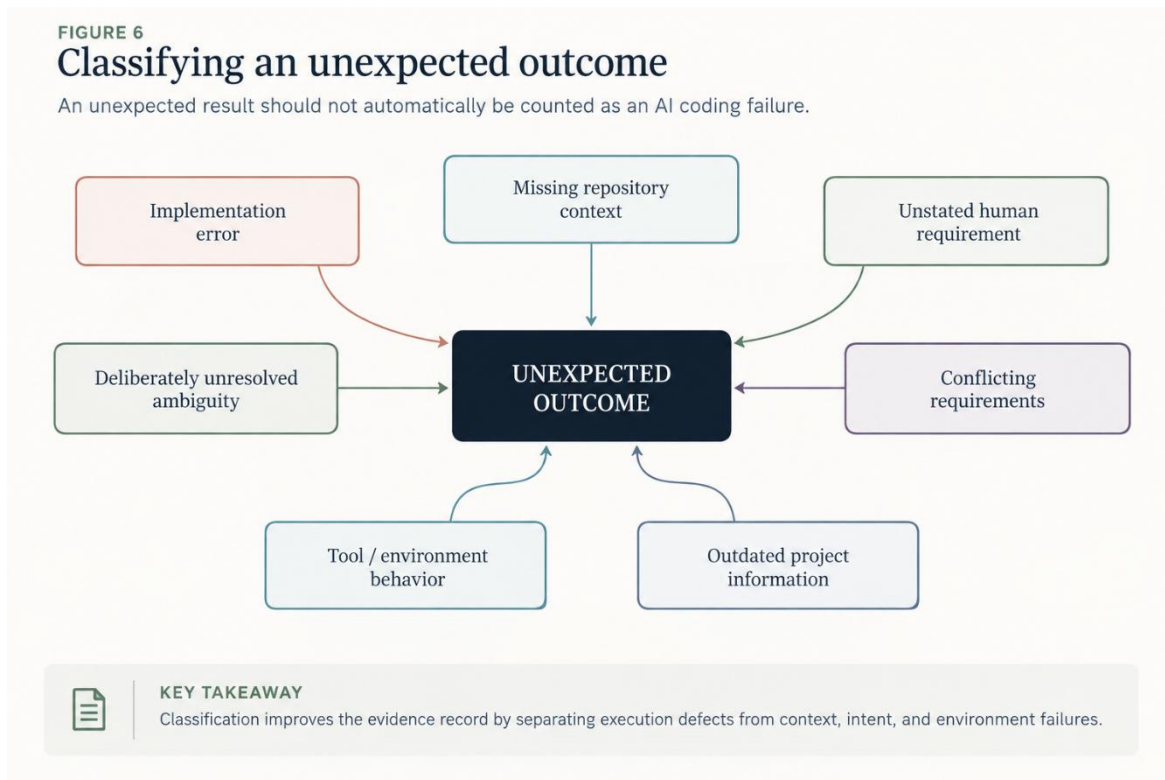


Figure 6. Classification model for unexpected AI-assisted development outcomes.

53. Candidate Metrics for the Continuing Experiment

The next phase of EpochLex could reasonably collect a small number of recurring measures without turning the project into a formal laboratory study. These metrics should remain descriptive until enough observations exist to support statistical interpretation. A dozen Codex tasks would provide useful case material but would still be a small sample for comparing model performance. The goal should be longitudinal evidence rather than prematurely producing rankings among tools.

Metric	What it could tell us
First-pass acceptance	How often bounded AI implementation is accepted without substantive revision
Corrective follow-up rate	How often a merged change requires a later defect fix
Human-review findings	What classes of problems are still found primarily by human review
Ambiguity identification	Whether AI recognizes missing product decisions before implementing them
Token usage by task type	Relative interaction cost of research, implementation, debugging, documentation, and QA
Human review effort band	Approximate human attention required after AI implementation
Validation coverage	Which checks were actually performed before merge
Model/tool used	Whether behavior appears to change across systems
Issue-to-PR time	How quickly bounded work moves from definition to reviewable implementation
Documentation follow-up	How often implementation makes existing project documentation inaccurate

54. What Can Be Quantified Now

At the September 18 snapshot, the experiment can confidently quantify repository activity, corpus growth, provenance status, issue state, several large migration operations, and the dates of major workflow transitions. The experiment can also reconstruct a meaningful amount of historical behavior from pull-request sequences, including when major features generated corrective work and when product expansion shifted the project's operational bottleneck.

Human effort remains unmeasured, while exact historical token totals and model-level identities were not recoverable from the retained evidence for this edition. These values are therefore reported as unavailable rather than estimated, and any future additions should preserve the distinction between measured, reconstructed, and interpretive evidence.

Productivity improvement, cost savings, and relative superiority to conventional development remain outside the supported evidence. Those questions would require either a comparative baseline or substantially more detailed labor and cost records than the initial experiment collected.

55. What the Numbers Tell Us

The numbers reinforce several lessons from the case studies, but they do not replace the qualitative record.

The growth from 27 to 365 entries shows substantial throughput. The shift from 122 researched provenance records to 243 pending records shows the downstream research pressure that followed. Completing 61 provenance records then shows that the workflow could be redirected toward consolidation when that imbalance became significant.

The 98-pull-request history shows repeated iteration, but follow-up corrections prevent that count from being read simply as feature production. The more useful picture is a development system in which implementation, observation, correction, documentation, and maintenance happened in rapid succession.

The issue record similarly shows that unresolved work increasingly involves areas where automation has limits. Accessibility observation, pronunciation quality, source evaluation, and editorial consistency remain partly dependent on human review.

The numbers point to the same pattern as the case studies: AI increased the capacity to produce and revise project artifacts, but it did not remove the rest of the product-development system. It changed which parts of that system became most important.

The next stage should improve the measurements without rewriting the first stage as something more precise than it was. Data collected during the ChatGPT + Codex + GitHub phase can make later comparisons clearer while preserving the first phase as an accurate account of how the experiment began.

Part V — Discussion, Lessons, and Implications

56. The Experiment Was Not Really About Whether AI Could Write Code

The original question became too narrow almost immediately. By 2026, showing that an AI system could generate HTML, CSS, JavaScript, documentation, or structured data was not especially informative by itself. The more useful question was whether AI could participate meaningfully in the continuing development of a product whose requirements, architecture, editorial standards, and quality expectations were still evolving.

EpochLex shows AI participating across a much broader portion of the lifecycle than code generation alone. AI assisted with product exploration, interface concepts, architecture analysis, research organization, debugging, documentation, quality assurance, source discovery, corpus expansion, maintenance, and implementation. That breadth matters because many of the difficult decisions were not syntax problems.

The more consequential questions involved matters such as whether provenance should be separated from dictionary content, whether a term deserved a distinct entry, whether a new feature justified additional infrastructure, whether an implementation matched the intended reader experience, or whether a source supported the strength of an attribution. AI could contribute to those discussions, but the project repeatedly required human judgment to resolve them.

The experiment became less about whether AI could produce software and more about what happens when producing software becomes easier. Once implementation is no longer the dominant constraint, architecture, requirements, editorial judgment, testing, maintenance, and product direction become more visible parts of the work.

57. AI Reduced the Cost of Trying an Idea

The strongest practical advantage observed during the first phase of EpochLex was the reduction in effort required to turn an idea into something that could be evaluated. A feature did not always need to be specified exhaustively before implementation because a working version could often be produced quickly enough to serve as part of the design process. That implementation could then be accepted, rejected, refined, or used to expose a missing requirement.

This changed the relationship between planning and implementation. Traditional development often creates economic pressure to settle more questions before code is written because implementation has historically represented a significant investment. In EpochLex, some decisions could reasonably be deferred until a functional version made the consequences easier to understand.

That does not mean planning became unnecessary. Features that affected architecture, canonical data, security, privacy, or other high-consequence areas still benefited from explicit decisions before implementation. The difference was that lower-risk product questions could sometimes be explored through working software rather than through increasingly detailed speculative requirements.

The PWA experience demonstrates this distinction particularly well. It was difficult to define every characteristic of an appropriate installed EpochLex interface before one existed. Once the initial PWA could be used, the difference between a technically installable website and an intentional application experience became much easier to describe.

58. Faster Implementation Changes the Economics of Iteration

EpochLex suggests that one of the most important effects of AI-assisted development is a change in the economics of iteration. If a prototype, interface revision, migration, or data transformation requires substantially less implementation effort, a project can examine more alternatives before committing to one.

That creates obvious benefits, but it also creates a new temptation. When implementation is inexpensive, it becomes easy to continue building simply because additional work can be generated quickly. A project can accumulate features, abstractions, documentation, or data faster than the human participants can evaluate their long-term value.

EpochLex encountered this most clearly during corpus expansion. Increasing the publication rate from 122 to 365 entries demonstrated that structured AI-assisted workflows could produce a large amount of useful material, but the resulting provenance backlog immediately exposed a slower part of the system. Continuing to increase the corpus simply because additional entries could be produced would have made the imbalance larger.

The ability to generate more work is not evidence that more work should be generated. AI increases implementation capacity, but product direction still determines whether that capacity is being applied to the right problem.

59. The Hammer and the Nail Gun

A useful analogy is the difference between driving nails with a hammer and using a nail gun. The nail gun can dramatically increase the number of nails that can be driven in a given period, but it does not determine where the wall belongs, whether the framing is square, whether the correct material is being used, or whether a nail should be driven at all. Increased speed amplifies the consequences of both correct and incorrect decisions.

AI-assisted development operated similarly during the EpochLex experiment. Repository-wide changes, corpus additions, documentation revisions, and interface implementations could be produced rapidly. When the underlying decision was sound, that speed was valuable. When an assumption was incorrect or incomplete, the same speed could propagate the problem across many files or records.

This is why traditional development disciplines did not become less important as AI capability increased. Architecture, requirements, validation, source control, documentation, and review provide the structure that determines where accelerated implementation should be applied. The faster the implementation tool becomes, the more consequential those upstream decisions can become.

The analogy should not be interpreted as reducing AI to a simple mechanical tool. Modern models participate in reasoning, analysis, research, debugging, and planning in ways a nail gun obviously does not. The useful part of the comparison is narrower: increasing production capacity does not remove the need to understand what is being produced.

60. Established Development Practices Did Not Disappear

The early language around vibe coding can create the impression that natural-language development replaces formal software practice. The EpochLex history does not support that view. The project continued to use conversational direction while becoming more dependent on explicit architecture, testing, documentation, review, and source-control discipline.

Architecture documentation became more important as new features approached the boundaries of the static site model. QA guidance expanded after mobile, pronunciation, accessibility, and caching problems demonstrated the limits of informal testing. Issues and pull requests became more structured as the project moved toward open-source participation, while repository instructions became necessary once Codex was introduced into the workflow.

This evolution does not mean the experiment gradually abandoned vibe coding. The human interface to the development process remained substantially conversational. What changed was the amount of durable context available behind that conversation.

The later EpochLex workflow points to a more useful interpretation of mature vibe coding: natural language can serve as an interface to a disciplined development system rather than as a substitute for one.

61. Repository Context Became a Form of Organizational Memory

One of the more important changes during the observation period was the migration of knowledge from conversation into the repository. Early in the experiment, recent chat history contained much of the information necessary to understand why a decision had been made. That approach became less reliable as the number of features, research rules, contributors, and development tools increased.

Documents such as `PROVENANCE.md`, `docs/QA.md`, `docs/ARCHITECTURE.md`, `CONTRIBUTING.md`, and later `AGENTS.md` gradually became a form of organizational memory. They allowed future work to begin from

established decisions without requiring the AI or a contributor to reconstruct those decisions from earlier conversations.

This matters especially in AI-assisted development because models do not carry a continuous human memory of the project. Context has to be supplied, retrieved, or encoded somewhere the system can access. Accurate architecture and decision documentation therefore supports human maintainers and improves the environment in which future AI systems work.

The Codex touch-target work demonstrates this value. Codex could compare an old pull request with current implementation and current project constraints rather than treating the earlier solution as authoritative. That behavior depended partly on the repository having enough durable context to distinguish an old implementation from the current product state.

62. Documentation Is Part of the AI Control System

Documentation in EpochLex is not only explanatory material for people who want to understand the project. It increasingly acts as part of the control system for AI-assisted development.

A statement in AGENTS.md that the project should not introduce a backend merely because it is a common architectural pattern constrains future implementation. A provenance rule stating that AI output is not a source prevents generated historical claims from acquiring authority merely because they are plausible. QA documentation tells future agents which behaviors need validation and which limitations cannot be proven through repository checks alone.

An undocumented lesson depends on the human remembering it or on the AI independently recognizing the same issue. A documented lesson can become part of the context supplied to future implementation work.

This provides another reason to take documentation drift seriously. When documentation is being used as input to AI development, stale documentation is no longer merely inconvenient. It can actively cause future agents to make decisions based on an obsolete model of the project.

63. AI Can Accelerate Technical Debt as Easily as Features

The EpochLex record contains several examples where AI's ability to perform broad changes was beneficial. Hundreds of records could be migrated, URLs could be normalized across many files, and documentation could be synchronized quickly. The same capability would make an incorrectly scoped migration equally efficient.

This changes the risk profile of some maintenance operations. A repetitive manual change may be slow but naturally limited by human attention. An AI-assisted repository-wide change can affect hundreds of files almost immediately, making validation more important rather than less.

The later PRs increasingly reflected this understanding. Large repetitive changes were accompanied by checks demonstrating not only that the desired new state existed but also that unrelated editorial content had not changed. PR #133, for example, validated that the legacy URL replacement affected only the intended public URL base.

This pattern suggests that AI-assisted maintenance should emphasize invariants: for a large automated change, it may be just as important to define what must remain unchanged as to describe what should change.

64. Human Judgment Became the Scarcer Resource

Implementation capacity increased substantially during the experiment, but human review did not scale in the same way. Every significant product change still required someone to decide whether the result was appropriate, whether a definition was accurate, whether a source was credible, whether a user experience felt right, or whether an architectural tradeoff was acceptable.

This became particularly visible in provenance research. AI could locate possible sources and organize material rapidly, but a credible reference work still required a human to determine whether an origin claim was justified. The same problem appeared in interface design, where a technically functioning control could still feel inappropriate in the broader product experience.

Human work did not become less important; it became more concentrated around judgment. When production becomes easier, evaluation can become the scarce resource that determines how much generated work can responsibly be accepted.

That observation may explain why some remaining EpochLex issues involve keyboard review, pronunciation quality, source quality, and editorial consistency rather than unfinished implementation. These tasks require forms of evaluation that are not fully reducible to code generation.

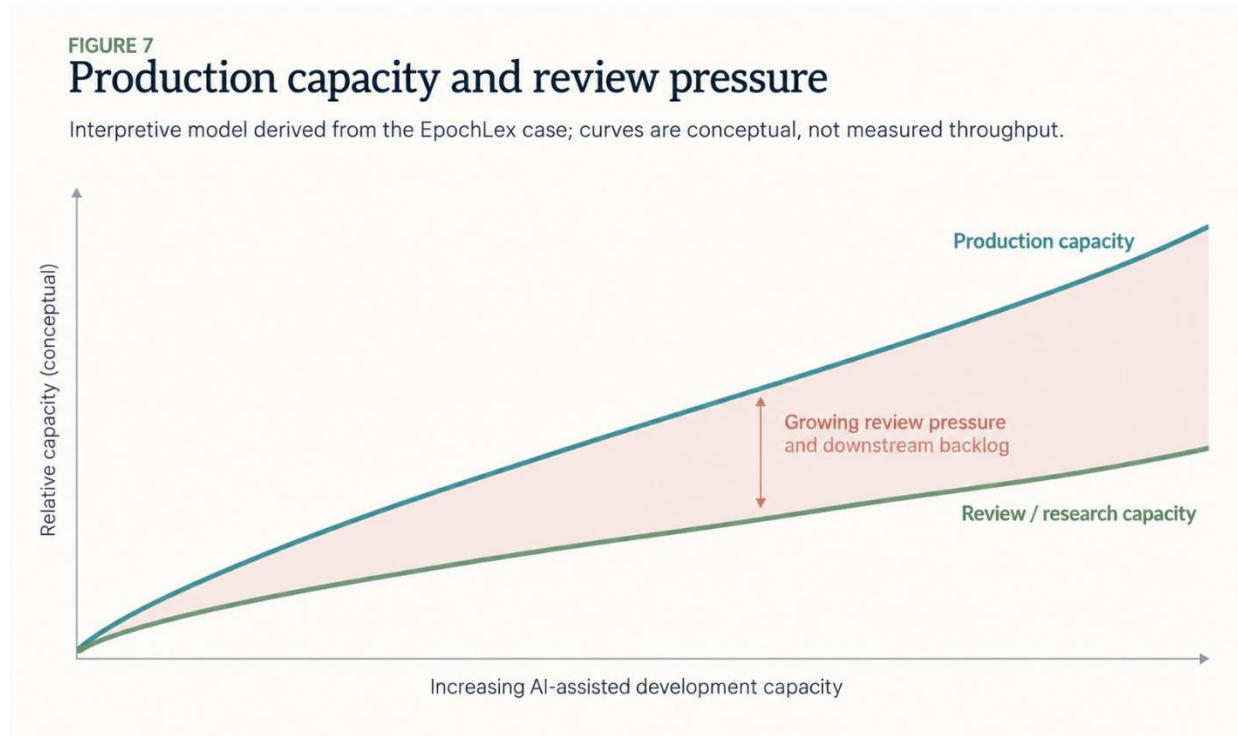


Figure 7. Interpretive model of increased production capacity and the resulting pressure on review and research.

65. Human Review Is More Than Error Checking

It would be incomplete to describe the human role only as checking whether AI output contains mistakes. Many of the most consequential EpochLex decisions involved alternatives that could all have been technically valid.

The AILex-to-EpochLex rename did not correct a syntax error. The decision to preserve the static architecture when implementing Word of the Day was not a debugging task. Separating publication readiness from provenance completion involved defining the editorial operating model of the dictionary.

Human review therefore included product ownership, prioritization, taste, risk assessment, evidence evaluation, and accountability. AI could contribute analysis or alternatives, but someone still had to decide what the project would optimize and what it was willing to accept.

This distinction is important when discussing the possibility of more autonomous development systems. An agent may eventually become much better at generating, testing, and repairing implementation, but greater technical autonomy does not automatically answer who should determine product intent or accept responsibility for public claims.

66. Ambiguity Is Shared Between the Human and the AI

The ambiguity case complicates simple descriptions of AI error. A model can misinterpret a clear requirement, but a human can also believe a requirement was clear when an important assumption never left the human's head.

AI-assisted development makes this problem unusually visible because the implementation arrives quickly enough to function as a mirror of the specification. When the result feels wrong, examining the discrepancy can reveal whether the model ignored an instruction or whether the instruction never existed.

This makes ambiguity a shared responsibility. The AI system should ideally recognize decisions that exceed safe implementation discretion, while the human should recognize that familiarity with the product can make implicit expectations feel more obvious than they are.

EpochLex benefits from separating defects caused by incorrect execution from defects caused by incomplete intent. Both require correction in a production product, but they point to different improvements in the development process.

67. The Ability to Surface Ambiguity May Become a Model Capability Worth Measuring

As AI development systems become more capable, generating a plausible implementation may become less impressive than recognizing when implementation should stop. A model that identifies an unresolved architectural question or conflicting product assumption before writing code may provide more value than one that always chooses a plausible interpretation and proceeds.

That suggests a useful future measure for EpochLex: record when an AI identifies missing information, separates an implementation detail from a product decision, or asks for clarification because several materially different outcomes are possible.

The metric would not reward indiscriminate questioning. An agent that asks the human to decide every trivial CSS value would simply move implementation work back to the user. The useful behavior is selective recognition of ambiguity whose consequences are significant enough to require human judgment.

Over time, EpochLex could compare whether different models and workflows handle this boundary differently. Such evidence could prove more meaningful than comparing which system generates the most code.

68. Product Responsibility Limits Experimental Freedom

Because EpochLex is both an experiment and a public product, experimentation cannot be the only priority. There are circumstances in which preserving ambiguity or allowing a known failure to recur would provide interesting evidence but would not be responsible product behavior.

Security, privacy, data integrity, misleading public claims, source credibility, and destructive operations fall clearly into this category. Once a meaningful risk is recognized, the project should resolve it rather than leaving it open for another model to encounter independently.

The same principle applies where accessibility barriers materially prevent readers from using the product. An unresolved touch-target preference may initially produce useful evidence, but once testing establishes that controls are impractically small, retaining that ambiguity would provide little additional value.

Experimental freedom therefore has a product-safety boundary. EpochLex can preserve uncertainty only where the cost of being wrong is low, reversible, and transparent.

69. Architecture Became a Decision Framework

The static architecture began as a simple solution appropriate to the original dictionary. Over time, it evolved into a framework for deciding how much complexity a feature was allowed to introduce.

Word of the Day fit naturally within that framework because it could derive from existing source-of-truth data. Push notifications did not, because they would require persistent subscriptions and trusted server-side credentials. The architecture helped distinguish a feature that fit cleanly from one that would change the project's operating model.

This does not imply that architecture should never change. A rigid architecture that prevents valuable product development can become as harmful as unnecessary complexity. The useful principle is that architectural change should be an explicit tradeoff rather than an accidental side effect of generating a feature.

AI-assisted development makes that discipline particularly important because models are capable of generating sophisticated infrastructure quickly. The existence of a technically credible implementation can make unnecessary complexity appear cheaper than it really is, while long-term operation, security, and maintenance costs remain outside the immediate prompt.

70. Simplicity Has Ongoing Value in AI-Assisted Systems

The EpochLex architecture also demonstrates another advantage of simplicity: it makes the project easier for both humans and AI systems to inspect. Canonical data is stored in source-controlled files, public behavior can be examined without a complex deployment stack, and significant decisions are visible in repository history.

This inspectability improves review. When a feature changes, the project owner can reason more directly about what changed and why. An AI agent can also search a smaller number of established patterns rather than navigating multiple layers of infrastructure.

Simplicity should not be romanticized. The static architecture creates real limitations, and future requirements may justify additional services. Its value during the experiment was that complexity had to earn its place rather than arrive by default.

That principle may generalize beyond EpochLex. AI reduces the implementation cost of architectural complexity, but it does not necessarily reduce the long-term cognitive and operational cost of maintaining that complexity.

71. AI-Assisted Research Has Different Risks From AI-Assisted Coding

The project also demonstrates that AI assistance cannot be evaluated as one uniform activity. A coding error and a research error have different consequences and require different safeguards.

Incorrect code often produces an observable failure that can eventually be reproduced, tested, and corrected. An incorrect historical attribution may look completely plausible while remaining false. If the claim is then repeated elsewhere, the error may become more difficult to recognize rather than easier.

EpochLex therefore treats AI-generated research as assistance rather than evidence. Models can help identify sources, compare wording, organize timelines, and suggest questions, but the supporting claim must ultimately be grounded in an external source appropriate to that claim.

This distinction matters especially for a reference work whose credibility depends on what it says, not only on whether the interface works. AI-assisted research may therefore need stronger provenance controls than AI-assisted implementation because a plausible error can look authoritative.

72. The Meaning of “AI-Built” Requires Precision

EpochLex could reasonably be described informally as being built with AI, but that phrase can conceal more than it explains. AI was materially involved throughout the development process, yet the project was not independently conceived, governed, researched, approved, or released by an autonomous system.

The repository's description—human-directed, AI-assisted, and human-reviewed—is more precise. It recognizes substantial AI involvement without erasing human accountability.

This distinction becomes more important as outside contributors participate. A future EpochLex contribution may be written entirely by a person, generated substantially by another model, created with a different coding agent, or produced through a combination of tools. The project can remain AI-assisted without requiring every contribution to share one development method.

The report should therefore avoid turning a changing development environment into one fixed technology claim. EpochLex is useful as an experiment partly because its toolchain can change while its product principles remain more stable.

73. Tool Changes Complicate Cause and Effect

The introduction of Codex demonstrates why later improvements cannot automatically be attributed to one cause. By September 17, Codex was working within a repository that had already accumulated architecture documentation, QA practices, issues, design rules, editorial standards, and nearly three weeks of implementation history.

If later Codex-assisted work appears more precise than some August implementation, several explanations are possible. Codex may be better suited to repository work, the underlying model may be more capable, the task may be more tightly bounded, the repository may provide better context, or the human may simply have become better at directing AI development.

Those factors are not isolated in the first phase of EpochLex. The report can document the workflow change and compare individual cases, but it cannot responsibly assign a percentage of improvement to Codex, a model change, repository maturity, or human learning.

More deliberate measurement may improve that picture. If future tasks record model, task type, requirements clarity, token usage, validation, human intervention, and follow-up correction, later phases may support more specific comparisons.

74. The Work Is Shifting Rather Than Disappearing

The EpochLex experience suggests that AI changes what the person directing the work spends time doing rather than simply removing that person from the process. Less time may be required to write every implementation detail manually, while more attention moves toward describing desired behavior, evaluating generated solutions, maintaining project context, designing validation, and deciding where automation should stop.

For designers, technologists, developers, and other professionals, this can broaden the range of work that is practical to explore directly. Ideas that once required substantial coding effort or a separate development team can be tested much earlier. Technical expertise still matters because architecture, security, maintainability, and failure modes do not disappear when implementation becomes easier.

The shift may favor people who can connect domains that were previously more separated. Product thinking, design judgment, technical understanding, research discipline, and communication all become important when natural language becomes one of the main interfaces through which digital work is created.

This does not imply that one person should replace a multidisciplinary team in every context. EpochLex is a relatively bounded public reference project, and its risk profile differs substantially from financial systems, healthcare software, critical infrastructure, or applications handling sensitive personal data.

75. Implications for Professional Practice

The lessons from EpochLex are not limited to software development. Many industries have already experienced similar shifts when a faster production tool changed how work was created, reviewed, and coordinated. Architecture, engineering, and construction provide one useful comparison because the progression from hand drafting to CAD, BIM, automation, and scripting repeatedly increased production capacity without removing the need for standards, coordination, or professional judgment.

Automation illustrates the pattern clearly. Dynamo, scripts, templates, shared parameters, model-checking tools, and similar workflow accelerators can reduce repetitive work dramatically, but they can also distribute a bad assumption across an entire project more efficiently than a manual process. The same principle applies well beyond AEC: faster production increases the value of clear rules, review, and accountability.

AI-assisted development follows that same broad pattern. The implementation mechanism may change substantially, but familiar questions remain: What problem are we solving? What constraints govern the work? What information is authoritative? How will the result be checked? What happens when an assumption is wrong? Who is responsible for accepting the result?

Seen this way, AI does not make existing process knowledge obsolete. Much of that knowledge becomes more valuable because increased production speed raises the cost of weak standards and unclear intent.

76. Process Knowledge May Outlast Tool Knowledge

Software tools change rapidly, and the AI systems used in the EpochLex experiment were already changing during the initial observation period. It would therefore be difficult to build a durable methodology around the assumption that one particular model or development environment will remain dominant.

Process principles are more portable. Current repository state should take precedence over stale planning material. Claims should be supported by evidence appropriate to the claim. High-risk ambiguity should be resolved rather than inferred. Large automated changes should preserve clearly defined invariants. Validation should state both what was tested and what was not.

These principles remain useful even if ChatGPT, Codex, or the underlying models change. They also remain relevant if a contributor uses a completely different toolchain. The case study therefore suggests that durable AI-development practice may depend less on mastering one model interface and more on learning how to structure context, requirements, evidence, validation, and human decision boundaries around systems that will continue to change.

77. Vibe Coding and Professional Practice Are Not Necessarily Opposites

The EpochLex experiment began partly in the spirit commonly associated with vibe coding: describe what is wanted conversationally, allow AI to implement a substantial portion of it, evaluate the result, and continue iterating. As the project matured, that workflow did not disappear.

What changed was the professional structure around it. The same conversational interface began operating against documented architecture, established editorial standards, issue definitions, QA expectations, source-controlled data, and scoped pull requests.

This suggests that the lasting distinction may not be between vibe coding and professional development. A more useful distinction is between AI-assisted work grounded in reliable project context and AI-assisted work that depends mainly on whatever information happens to be present in the current conversation.

EpochLex gradually moved toward the former while retaining the speed and accessibility of conversational direction. That hybrid may be a more realistic description of how AI development practices mature than the idea that projects must eventually abandon natural-language workflows in favor of traditional methods.

78. What the Experiment Suggests So Far

The first phase of the EpochLex experiment supports several conclusions with reasonable confidence. AI can materially participate across a broad product lifecycle, not only code generation. It can substantially reduce the effort required to move from an idea to something concrete enough to evaluate, and that capability can accelerate both useful discovery and the propagation of incorrect assumptions.

The experiment also shows that implementation speed does not remove the need for architecture, source control, QA, research standards, documentation, and human review. Those controls became more explicit as the project grew. The largest operational problems increasingly involved judgment, research, validation, and accumulated project knowledge rather than an inability to generate code.

The record also supports a more nuanced view of failure. Some problems were implementation defects, some were deployment or cache-state problems, some were documentation drift, and some revealed requirements that had never been articulated. Calling all of them “AI wrote bad code” would erase much of what the experiment can teach.

The strongest overall conclusion at this stage is not that AI replaces software-development practice. It is that AI changes the rate at which product intent can become implementation, which in turn changes where development effort and risk accumulate.

79. A More Mature Statement of the Experiment

At the beginning, the EpochLex experiment could have been summarized with a relatively simple question: could the Dictionary of the AI Era itself be built using AI? The repository history now supports a more useful formulation. The question is how far a serious public reference project can be designed, researched, implemented, tested, documented, corrected, and maintained through human-directed AI collaboration while preserving human accountability for the product and its claims.

That question acknowledges that success cannot be measured only by whether the site works. A successful experiment must also account for the accuracy of the dictionary, the integrity of the research process, the maintainability of the architecture, the usability of the product, and the transparency of the development method.

The question also leaves room for the answer to change. New models, Codex, outside contributors, and future development practices may alter what AI can perform reliably. The report is therefore a record of observed development, not a permanent verdict on the limits of the technology.

80. The Central Lesson

The EpochLex experiment began with the expectation that AI might make it possible to build a useful product more quickly. The evidence supports that expectation in a broad qualitative sense, even though the project cannot yet quantify the productivity difference against a conventional development baseline.

What became more important was the realization that faster production magnifies the value of direction. When implementation is expensive, poor decisions may move slowly. When implementation becomes inexpensive, both good and bad decisions can propagate quickly.

The practical consequence is that judgment, architecture, standards, validation, and documentation do not become obsolete in an AI-assisted workflow. They are the mechanisms that direct increased implementation capacity.

EpochLex therefore suggests that the defining skill in AI-assisted development may be less about making an AI produce more and more about deciding what should be produced, what must remain unchanged, how the result should be evaluated, and when the system should stop and return a decision to the human.

Part VI — Conclusion, Limitations, and the Continuing Experiment

81. The September 18 Snapshot

This report documents the first major phase of the EpochLex experiment through a repository snapshot taken on September 18, 2026. At that point, EpochLex had grown from an initial 27-entry dictionary into a 365-entry public reference work with structured provenance, dedicated term pages, search and browsing tools, pronunciation, related-term discovery, publishing metadata, a Progressive Web App, Word of the Day, contributor infrastructure, quality-assurance guidance, and a documented editorial and architectural model.

The development process had also changed substantially. The earliest work relied heavily on conversational direction through ChatGPT combined with GitHub-based implementation and review. Over time, more project knowledge moved into source-controlled documentation, issues, pull requests, validation procedures, and explicit project constraints. On September 17, Codex was added to that existing ChatGPT and GitHub workflow as a more implementation-focused development agent.

The snapshot captures EpochLex at an important transition rather than an endpoint. The product had matured enough to support meaningful analysis of the first development phase, while the introduction of Codex created a natural boundary for a more deliberately measured second phase. Future reports should build on this record rather than rewrite the first phase in light of later capabilities.

82. What the First Phase Established

The first phase demonstrates that AI can participate materially across a broad range of product-development activities. The repository record contains AI-assisted work involving product exploration, interface implementation, structured data, debugging, documentation, research organization, quality assurance, accessibility, publishing, maintenance, and editorial support. The breadth of involvement is more significant than the fact that AI produced code, because many of the difficult project decisions were not primarily coding decisions.

Human direction also remained consequential throughout the process. Product identity, architecture, editorial policy, source evaluation, taxonomy, interface quality, feature prioritization, ambiguity resolution, and release judgment were repeatedly shaped by human decisions. The development record does not support describing EpochLex as an autonomous AI-created project.

The project further demonstrates that faster implementation does not remove the need for conventional development practices. Architecture, source control, QA, documentation, issue tracking, validation, and human review became more explicit as the project grew. The increasing sophistication of the AI workflow was accompanied by more development discipline rather than by the disappearance of that discipline.

Finally, the first phase established that failure in AI-assisted development is not one category. Some problems resulted from incorrect implementation, some from caching or deployment state, some from stale documentation, some from incomplete repository context, and others from human assumptions that had never been converted into explicit requirements. That distinction should remain central to future analysis.

83. Limitations of the First Report

The largest limitation is the absence of a control project. EpochLex was not built twice under identical requirements, once conventionally and once through AI-assisted development. The report can document what happened and how quickly project state changed, but it cannot establish how much faster AI made the work compared with a matched conventional process.

Human labor was also not tracked systematically during the first phase. Conversation timestamps, pull-request timing, and repository activity can identify periods of work, but they cannot determine how many hours were spent planning, reviewing, researching, testing, or working outside the recorded AI interactions. This prevents credible calculations of labor savings, effective hourly productivity, or professional-services cost equivalence.

Historical token usage is another unresolved measurement. A substantial portion of the ChatGPT interaction record may be available for reconstruction, and recent Codex or API activity may provide direct analytics, but the experiment did not begin with project-specific token accounting. Any historical lifetime estimate must therefore distinguish visible transcript tokens, modeled context-processing estimates, and directly measured usage.

The AI model timeline is also incomplete. The product-level tools used during different phases are better established than the exact underlying model used for every historical interaction. Model identity is therefore documented only where it can be verified rather than reconstructed from general release dates.

The repository record itself has limitations as evidence. A pull request shows what changed, but it does not always preserve every rejected idea, informal test, design reaction, or conversation that led to the change. Development conversations help address that gap, but neither source constitutes a complete forensic record of every decision or AI interaction.

84. Limits on Generalization

EpochLex is one project with a particular combination of subject matter, architecture, risk, and human direction. It is a public reference product based primarily on static web technology and source-controlled data. Its findings should not be assumed to apply directly to financial systems, regulated health information, safety-critical infrastructure, large-scale authentication, confidential corporate data, or other substantially different risk environments.

The project owner also brings existing experience in design, technology, software tools, technical workflows, and structured project thinking. The experiment therefore does not establish what a person with no relevant technical or product background would achieve using the same AI systems. Human skill remains one of the variables in the experiment rather than a neutral constant.

EpochLex also cannot establish whether AI-assisted code is better or worse than code produced by professional software developers. No matched conventional codebase exists for comparison, and many project outcomes depend on product and editorial decisions that cannot be reduced to code quality alone. This report is better understood as a case study of one human-AI development system than as a benchmark against professional engineering teams.

85. Codex Marks a New Observation Period

The addition of Codex on September 17 creates a useful boundary for the next phase because the workflow can now be measured prospectively rather than reconstructed entirely after the fact. ChatGPT remains part of the development environment for planning, product discussion, research, design reasoning, and requirements. Codex adds a repository-oriented implementation path operating under explicit project instructions, while GitHub remains the source-control, review, and historical record.

The first Codex-assisted tasks already suggest useful research questions. PR #128 demonstrated extensive repository inspection and validation, while PR #129 demonstrated that strong automated testing did not eliminate

the need for human mobile and PWA verification. These examples are informative, but the sample remains too small to establish whether Codex changes defect rates, review effort, implementation quality, or productivity.

The next observation period should collect structured evidence rather than assume the new workflow is better. A useful experiment must leave room for the possibility that a new tool improves some parts of the work while making little difference—or adding complexity—in others.

86. Measurement Plan for the Next Phase

Beginning with the ChatGPT + Codex + GitHub phase, substantial tasks should receive a lightweight experimental record. The goal is not to create administration around every commit, but to capture information that would be difficult or impossible to reconstruct later.

For each meaningful task, the project should record the originating requirement or issue, task category, AI tool or tools used, model when verifiable, important constraints supplied to the implementation, validation performed, human-review findings, and final disposition. Approximate token usage and human-review effort should also be captured when those values are available without disproportionate effort.

Unexpected results should be classified according to cause rather than being labeled automatically as AI failures. At minimum, the continuing experiment should distinguish implementation errors, missing project context, unstated human requirements, conflicting requirements, outdated documentation, environment or caching behavior, and intentionally unresolved ambiguity.

Over time, this data could support more useful comparisons between phases. The project could examine first-pass acceptance, corrective follow-up rates, ambiguity detection, human-review effort, token use by task category, validation coverage, and the types of problems discovered only through real-world testing.

87. Token Utilization Status

The evidence pass for this edition did not recover a complete project-specific token ledger or enough historical transcript and context data to calculate a defensible lifetime token total. The report therefore does not publish an exact or estimated total for the first observation period. If complete conversation exports or project-attributable analytics become available later, they can be added using the method in Appendix D without changing the conclusions already supported by repository evidence.

Any future token measurement should remain descriptive. Token volume can indicate the scale of human-AI interaction, but model differences, repeated context, caching, pricing structures, and subscription products make it unsuitable as a standalone measure of productivity, authorship, intelligence, or actual project cost.

88. Model Timeline Status

The retained record establishes the product-level toolchain but does not verify exact model variants or configurations for the historical ChatGPT work or the first Codex sessions. The timeline can state that ChatGPT was part of the maintainer workflow before September 17 and that Codex was added on September 17, while model-level fields remain intentionally unspecified.

That is an important distinction, not a blank to be guessed. A model being publicly available at the time does not prove it powered a particular interaction, so this report does not infer model use from release dates.

The limitation also prevents later improvements from being attributed too simply to process maturity or model capability. Repository context, task structure, AI tooling, human experience, and possibly model capability changed together, so the first observation period does not isolate their individual effects.

89. Contributors and the Expanding Experimental Environment

As EpochLex receives outside contributions, the experiment will become less centered on one maintainer workflow. Contributors may use ChatGPT, Codex, other commercial AI systems, local models, traditional editors, custom automation, or workflows that involve little or no AI assistance.

This variability should be treated as information rather than as a problem to eliminate. A genuinely open-source project should not require contributors to reproduce the maintainer's AI stack unless a specific technical requirement makes that necessary. Contribution quality should remain governed by the project's editorial, architectural, validation, and review standards rather than by allegiance to one development tool.

When contributors disclose material AI assistance, that information can become part of the experimental record. When tooling is not disclosed, the report should leave it unspecified rather than infer it from coding style or contribution speed.

If participation grows, EpochLex may eventually provide a useful environment for observing how different AI-assisted workflows interact with the same repository standards. That is a future possibility, not an established result of the first phase.

90. Ambiguity Should Continue to Be Observed Deliberately

The ambiguity finding also creates a reason not to over-standardize the next phase prematurely. Every meaningful product risk should still be resolved, but low-risk ambiguity can occasionally remain useful as a controlled observation.

The experiment should record when an AI identifies a missing decision before implementation, when it silently chooses among plausible alternatives, and when human review reveals an assumption that was never documented. These observations could eventually help distinguish systems that merely generate plausible implementations from systems that reason effectively about the boundaries of their authority.

Any deliberate preservation of ambiguity should remain low-risk, reversible, and documented. The project should not compromise security, privacy, data integrity, editorial credibility, accessibility, or architecture merely to create another experiment.

The goal is therefore not to maximize uncertainty. It is to preserve enough natural ambiguity to continue learning how AI systems interpret incomplete product intent without allowing that ambiguity to undermine the public product.

91. EpochLex as a Living Experiment

EpochLex describes itself as a Living Dictionary because its subject matter continues to change. The same description is increasingly appropriate for the development experiment.

The software, corpus, research process, development tools, and AI models will continue to evolve. A permanent verdict in September 2026 would misrepresent the project. A more useful approach is to define observation periods and compare later phases against the documented baseline.

This first report serves as the baseline. It records how the project began, how quickly it expanded, where problems emerged, what processes were created in response, and what remained unresolved when Codex entered the workflow.

A future report should not silently revise these findings based on later capabilities. It should ask how the next development phase differs from the first and whether new evidence changes any of the earlier interpretations.

92. Conclusion

EpochLex began with a relatively simple idea: collect and explain the rapidly changing language of the AI era. Within a short period, that idea became a functioning public reference work and, almost accidentally at first, a second experiment in how AI itself could participate in building the project.

The resulting development history demonstrates substantial capability. AI assisted with far more than writing code, and the project moved from a small initial dictionary to a much broader product with hundreds of entries, structured provenance, a researched editorial model, responsive interfaces, a PWA, contributor infrastructure, and increasingly explicit quality controls.

The same history also demonstrates substantial limitations. AI-assisted work produced regressions, missed product assumptions, encountered browser and cache behavior that required real-world testing, generated workloads faster than some forms of human review could be completed, and depended increasingly on accurate repository context. These limitations were not separate from the experiment; they were some of its most informative results.

The project therefore supports neither an uncritical claim that AI replaces conventional development nor the narrower idea that AI is simply a faster code generator. The change is more structural: AI reduced the effort required to move between an idea and an implementation, allowing more of the product to be explored through working software. That increased capacity made judgment, validation, architecture, evidence, and project context more consequential.

The experiment also suggests that many of the practices developed before generative AI remain relevant. Requirements still matter, even when they are expressed conversationally. Architecture still matters, even when an AI can generate a new service in minutes. Testing still matters, even when automated agents can execute broad validation. Source credibility still matters, even when a model can produce a plausible historical narrative almost instantly. What changes is the speed at which those decisions become executable.

The central question for the continuing EpochLex experiment is no longer whether AI can participate in building meaningful software. The repository already shows that it can. The more important question is how humans should direct, constrain, evaluate, and learn from that increased development capacity while remaining accountable for what the resulting product does and what it claims to know.

The September 18, 2026 snapshot does not close that question. It establishes the first documented baseline from which the next phase can be observed.

SUPPORTING MATERIAL

Appendices

Evidence registers, reconstruction methods, classification frameworks, and measurement definitions supporting the September 18, 2026 report snapshot.

Appendix A — Evidence Register

Appendix B — Repository Classification Framework

Appendix C — Model and Toolchain Timeline

Appendix D — Token Utilization Reconstruction

Appendix E — Next-Phase Measurement Framework

Appendix F — Report Terminology

Appendix G — Source Notes and Methodology Details

Appendix A — Evidence Register

The evidence register provides the traceable source layer for the report. Repository artifacts, pull requests, issues, development conversations, measurements, and direct analytics are distinguished so that findings can be tied to the kind of evidence that supports them. A September 18 verification pass reconfirmed 98 pull requests in the audited date range, 97 merged with PR #84 as the sole unmerged pull request, 30 issues with 25 closed and 5 open, and canonical datasets containing 365 entries with 183 researched and 182 pending provenance records. The frozen repository baseline is main after PR #133, merge commit 4b31946; later repository activity is outside this edition unless explicitly identified.

Evidence Classes

Code	Evidence type and use
R	Repository artifact — current source files, documentation, canonical datasets, architecture documents, or project standards.
PR	Pull request — implementation history, validation notes, scope descriptions, fixes, or changes in repository state.
I	Issue — problem definition, acceptance criteria, identified defects, research work, or contribution opportunities.
C	Development conversation — product intent, rejected approaches, design reactions, ambiguity, decision rationale, or human testing feedback.
M	Measurement — derived repository counts, corpus counts, issue counts, PR activity, token estimates, or other quantitative analysis.
A	Analytics record — direct product analytics such as Codex usage data or verified API token records when available.

Core Evidence Register

Evidence ID	Type	Date	Evidence	Primary use
PR-001	PR	Aug. 28	PR #1 — Initial Dictionary of the AI Era MVP	Initial architecture; 27-entry baseline
PR-005	PR	Aug. 28	PR #5 — Provenance, Sources & Term History Foundation	Provenance model; meaning vs. origin
PR-013	PR	Aug. 28	PR #13 — Accessible Mobile Navigation	Mobile/accessibility development
PR-015	PR	Aug. 28	PR #15 — Mobile Browse Regression Fixes	Regression and real-use correction
PR-023	PR	Aug. 28	PR #23 — 100-entry corpus completion	Initial corpus milestone
PR-030	PR	Aug. 29	PR #30 — Product Readiness Accessibility & QA Baseline	Formalization of QA
PR-031	PR	Aug. 29	PR #31 — EpochLex Brand Migration Foundation	Rename and broad migration
PR-032	PR	Aug. 29	PR #32 — EpochLex Mobile Cache Bust	Source correctness vs. delivered behavior
PR-036	PR	Aug. 29	PR #36 — Named-entity taxonomy expansion	Five-category taxonomy; entry types
PR-043	PR	Aug. 29	PR #43 — First researched named entities	Taxonomy applied to real cases
PR-045	PR	Aug. 29	PR #45 — DeepSeek entries	Editorial granularity and naming ambiguity
PR-047	PR	Aug. 29–30	PR #47 — EpochLex PWA foundation	Static architecture extended into PWA

Evidence ID	Type	Date	Evidence	Primary use
PR-059	PR	Aug. 30–31	PR #59 — Outside contributor readiness	Community infrastructure
PR-064	PR	Aug. 31	PR #64 — Community docs and public documentation	Discussions/Issues/Project/PR workflow
PR-082	PR	Sept. 5	PR #82 — Source-controlled candidate inventory	Persistent research planning
PR-083	PR	Sept. 5	PR #83 — Candidate research pipeline	Candidate status vs. publication approval
PR-084	PR	Sept. 5	PR #84 — Responsive touch targets	Stale implementation later superseded
PR-085	PR	Sept. 8	PR #85 — Separate publication and provenance research	Two-gate editorial model
PR-086	PR	Sept. 8	PR #86 — First pending-provenance publication batch	Start of major corpus expansion
PR-110	PR	Sept. 8	PR #110 — Corpus Batch Y	365-entry milestone; 243 pending
PR-111	PR	Sept. 8	PR #111 — Provenance review queue	Operational response to research backlog
PR-117	PR	Sept. 8	PR #117 — Provenance Batch 06	Completion of Wave 1
PR-118	PR	Sept. 9	PR #118 — Post-Wave-1 reassessment	Deliberate stop and state documentation
PR-119	PR	Sept. 15	PR #119 — Word of the Day	Feature within static architecture
PR-121	PR	Sept. 15	PR #121 — Standalone PWA shell	Installed-app intent becomes explicit
PR-122	PR	Sept. 15	PR #122 — PWA cache busting	Delivered-state correction
PR-127	PR	Sept. 17	PR #127 — Codex project instructions	Codex workflow introduction
PR-128	PR	Sept. 17	PR #128 — Responsive touch targets on current main	Structured Codex implementation
PR-129	PR	Sept. 17	PR #129 — Pronunciation and PWA script refresh	Human validation after extensive automated testing
PR-130	PR	Sept. 17	PR #130 — Cache-strategy documentation	Documentation synchronization
PR-131	PR	Sept. 17	PR #131 — Explicit entryType migration	Canonical data cleanup
PR-132	PR	Sept. 17	PR #132 — Migration documentation	Compatibility vs. canonical state
PR-133	PR	Sept. 17	PR #133 — Public URL normalization	Large repetitive maintenance operation
I-051	Issue	Aug. 30	Issue #51 — Keyboard-only QA	Remaining human QA work
I-052	Issue	Aug. 30	Issue #52 — Mobile/tablet audit	Real-device validation
I-053	Issue	Aug. 30	Issue #53 — Pronunciation quality	Human/device-dependent quality
I-056	Issue	Aug. 30	Issue #56 — Provenance source quality	Human research judgment
I-057	Issue	Aug. 30	Issue #57 — Canonical URL audit	Publishing integrity
I-060	Issue	Aug. 30	Issue #60 — Touch-target sizing	Accessibility requirement
R-001	Repository	Snapshot baseline	AI-TRANSPARENCY.md	Development model and AI disclosure
R-002	Repository	Snapshot baseline	PROVENANCE.md	Evidence standards and human review

Evidence ID	Type	Date	Evidence	Primary use
R-003	Repository	Snapshot baseline	docs/QA.md	Automated vs. human validation boundaries
R-004	Repository	Snapshot baseline	docs/ARCHITECTURE.md	Static architecture and boundary decisions
R-005	Repository	Snapshot baseline	AGENTS.md	Codex constraints and workflow
R-006	Repository	Snapshot baseline	CONTRIBUTING.md	Contributor and editorial requirements
C-001	Conversation	Aug. 28	AI Dictionary Terms discussion	Initial product/brand evolution
C-002	Conversation	Aug. 30	GitHub Projects and OSS readiness	Contributor-readiness rationale
C-003	Conversation	Sept. 15	PWA / Word of the Day / notifications	Product vs. architecture tradeoff
C-004	Conversation	Sept. 17	Codex project setup	New division of labor
C-005	Conversation	Sept. 17	Codex task review and mobile validation	Human validation after agent implementation
C-006	Conversation	Sept. 18	Ambiguity discussion	Ambiguity as experimental variable
M-001	Measurement	Sept. 18	98 PR / 97 merged count	Repository activity
M-002	Measurement	Sept. 18	27 → 100 → 365 corpus progression	Corpus scale
M-003	Measurement	Sept. 18	122/243 → 183/182 provenance progression	Bottleneck shift
M-004	Measurement	Sept. 18	PR activity by date	Development burst analysis
M-005	Measurement	Not reported	Historical token reconstruction	No defensible lifetime total from retained evidence
M-006	Measurement	Verified at product level	Model/toolchain reconstruction	ChatGPT/Codex transition established; exact models unspecified
A-001	Analytics	Not available	Codex usage analytics	No project-attributable analytics in evidence set
A-002	Analytics	Not established	API usage analytics	No project-attributable API record in evidence set

Evidence-to-Finding Map

The following map connects the report's principal findings to the strongest evidence anchors used in the first observation period.

Finding	Primary evidence
AI reduced idea-to-implementation distance	PR-001, PR-005, PR-023, C-001

Finding	Primary evidence
Human direction materially changed the project	PR-031, PR-036, PR-085, C-001, C-003
AI-assisted work produced real regressions	PR-015, PR-032, PR-122, PR-129
Human testing found issues automation did not	PR-128, PR-129, R-003, C-005
Faster production shifted the bottleneck	PR-085, PR-086, PR-110, PR-111, PR-117, M-003
Engineering discipline increased with maturity	PR-030, PR-059, PR-127, R-003, R-005
Architecture constrained implementation productively	PR-119, R-004, C-003
Repository context improved agent implementation	PR-127, PR-128, I-060
AI research required stronger provenance safeguards	PR-005, R-002
Ambiguity can expose unstated human requirements	PR-121, C-006, mobile/PWA development record
AI did not eliminate human judgment	R-001, R-002, PR-031, PR-085, PR-118
Codex introduction is a new variable, not a clean controlled experiment	PR-127, PR-128, C-004

Appendix B — Repository Classification Framework

Repository work is classified by primary work type so that feature development, corrective work, refinement, documentation, governance, and maintenance are not collapsed into a single activity count. Secondary tags can be added when a pull request spans a recurring area such as mobile, provenance, caching, or accessibility. This edition establishes the framework but does not assign every one of the 98 audited pull requests to a final category.

Primary Classification

Primary category	Definition
Product feature	New reader-facing functionality.
Corrective fix	Correction of behavior that did not meet the intended requirement or introduced an unintended defect.
Iterative refinement	Improvement to functioning behavior after use or design review reveals a preferable direction.
Editorial / data	Dictionary content, corpus, provenance, taxonomy, or research records.
Architecture / infrastructure	PWA infrastructure, caching, publishing, deployment, or structural behavior.
Documentation	Repository or reader-facing documentation without a material behavior change.
QA / accessibility	Testing infrastructure, accessibility correction, or repeatable quality assurance.
Governance / community	Contributor infrastructure, issue templates, contribution processes, or open-source operations.
Maintenance / migration	Schema normalization, stale URL cleanup, broad consistency updates, or similar technical maintenance.

Completed PR Classification

All 98 pull requests in the audited history were assigned one primary category according to the dominant purpose of the change. This is an analytical classification rather than a native GitHub label, and multi-purpose pull requests are represented by their primary intent. PR #84 remains in the classification even though it was closed without merge because it is part of the audited development history.

Primary category	Count	Pull requests
Editorial / data	50	#5–9, #18–23, #36, #43–45, #82–83, #85–117
Product feature	10	#1, #4, #10–12, #17, #24, #46, #119, #121
Corrective fix	8	#15, #25, #32, #48–50, #122, #129
Documentation	8	#29, #35, #64, #118, #120, #127, #130, #132
Iterative refinement	7	#3, #14, #16, #33, #123–125
Maintenance / migration	5	#28, #31, #126, #131, #133
QA / accessibility	4	#13, #30, #84, #128
Governance / community	3	#2, #34, #59

Primary category	Count	Pull requests
Architecture / infrastructure	3	#26, #27, #47

Secondary Tags

Secondary tags can identify recurring dimensions without replacing the primary classification. Useful tags for the current experiment include mobile, PWA, pronunciation, provenance, corpus, branding, accessibility, metadata, caching, Codex-assisted, documentation follow-up, post-merge correction, human-device finding, and automated-validation finding.

Appendix C — Model and Toolchain Timeline

The toolchain changed during the observation period and is therefore part of the experiment. The evidence pass verifies the product-level workflow—ChatGPT with the GitHub/repository process before September 17, then ChatGPT + Codex + GitHub/repository workflow from September 17—but does not recover exact historical model variants or configurations. Contributor tooling is recorded only when disclosed or otherwise directly established.

Period	Maintainer environment	Model evidence	Status
Aug. 28–Sept. 16, 2026	ChatGPT + GitHub/repository workflow	Exact ChatGPT model/configuration not recoverable from retained evidence	Product-level workflow established
Sept. 17 onward	ChatGPT + Codex + GitHub/repository workflow	Exact historical model/configuration not recoverable from retained evidence	Codex added to existing workflow
Contributor work	Variable by contributor	Recorded only when disclosed or directly established	No inference from absence of disclosure

Interpretive Boundary

Later improvements cannot be attributed to Codex or a model change in isolation. By September 17, the repository also contained more project context, clearer architecture, stronger quality-assurance guidance, and several weeks of accumulated human learning. Because exact model versions are not verified for this edition, model capability remains an unmeasured influence rather than a documented cause.

Appendix D — Token Utilization Reconstruction

EpochLex did not begin with project-specific token accounting. The evidence available for this edition is not sufficient for a defensible historical lifetime token total, so no exact or estimated total is reported. This appendix preserves the method to use if complete conversation exports or project-attributable usage analytics become available later.

Stage 1 — Conversation Identification

Identify a complete set of ChatGPT conversations materially related to EpochLex during the observation period and exclude conversations that mention the project only incidentally. A partial set should not be extrapolated into a lifetime total without an explicit sampling method.

Stage 2 — Visible Token Count

Tokenize recoverable user prompts, assistant responses, and relevant tool-visible material only when the transcript set is sufficiently complete. When the exact historical tokenizer is not known, use a documented approximation and preserve the uncertainty.

Stage 3 — Context Reconstruction

Estimate repeated context processing only where conversation structure makes the calculation reasonably reconstructable. A minimum and maximum range is preferable to a single precise-looking figure, and inaccessible internal processing should not be invented.

Stage 4 — Direct Analytics

Add directly measured Codex, API, or other usage analytics only when they can be attributed specifically to EpochLex and to the relevant observation period. No such project-attributable analytics are included in the evidence set for this edition.

Stage 5 — Reporting

Report visible reconstructed tokens, estimated historical processing, and directly measured usage as separate categories before presenting any combined range. If one component is unavailable, leave it unavailable rather than filling the gap with a nominal value. Token volume is treated as a measure of interaction scale, not as a standalone measure of intelligence, productivity, labor savings, authorship, or actual project cost.

Usage category	Status in this edition	Evidence
Historical visible ChatGPT conversation tokens	Not reported	Complete project transcript set not available in evidence package
Estimated historical model-processing range	Not reported	Insufficient basis without complete transcript/context reconstruction
Measured Codex usage	Not available	No project-attributable analytics in this edition
Measured API usage	Not established	No project-attributable API usage record in this edition
Combined experimental range	Not reported	Insufficient complete measured/reconstructed components

Appendix E — Next-Phase Measurement Framework

Beginning with the ChatGPT + Codex + GitHub phase, substantial development tasks can be recorded as they happen rather than reconstructed later. The framework is intentionally lightweight so measurement does not become a second project or materially change the behavior being observed.

Field	What it captures
Task ID	Stable identifier for the measured work item.
Date	When the task entered the measured workflow.
Issue / originating requirement	The requirement, issue, or decision that initiated the work.
Task category	Feature, corrective work, research, documentation, QA, maintenance, or other category.
AI systems used	ChatGPT, Codex, contributor tool, or other known system.
Verified model	Exact model only where the development record establishes it.

Field	What it captures
Human-defined constraints	Architecture, editorial, design, or risk boundaries supplied before implementation.
Known ambiguity at start	Any material uncertainty intentionally preserved or already recognized.
AI-identified ambiguity	Whether the AI surfaced a missing or conflicting decision before implementation.
Validation performed	Automated, browser, repository, device, accessibility, or editorial checks completed.
Human-review findings	Problems, tradeoffs, or missing requirements identified during review.
Corrective follow-up	Whether a later corrective pull request or change was required.
Post-merge regression	Whether a later problem was traced to the change.
Approximate token usage	Measured or reconstructed usage where available.
Approximate human review effort	Directional effort band rather than false minute-level precision.
Final disposition	Merged, revised, deferred, rejected, superseded, or other outcome.

Unexpected Outcome Classification

Unexpected results should be classified according to cause rather than counted automatically as AI coding failures. The current framework distinguishes implementation error, missing repository context, unstated human requirement, conflicting requirements, outdated project information, tool or environment behavior, deliberately unresolved ambiguity, and outcomes whose cause remains undetermined.

Appendix F — Report Terminology

The following terms are defined for consistency because common AI-development vocabulary is still unsettled and often carries different assumptions in different contexts.

AI-Assisted Development

Development in which an AI system materially supports planning, implementation, debugging, research, documentation, testing, analysis, or related work while human participants retain accountability for the project.

Human-Directed

The human participant determines project purpose, priorities, constraints, product direction, and acceptance decisions.

Human-Reviewed

AI-produced or AI-assisted work is subject to human evaluation before it is accepted as project output. Review may include code review, testing, editorial review, source evaluation, design judgment, or release decisions depending on the work.

Vibe Coding

For this report, vibe coding means conversational software development in which goals, requirements, feedback, and desired behavior are communicated substantially through natural language while AI systems generate or modify significant portions of the implementation. The term does not imply that output is accepted without testing or review.

Agentic Development

Development in which an AI system has greater responsibility for operating within a repository or tool environment, inspecting context, making bounded changes, running validation, and preparing work for human review. The term is used descriptively and does not imply complete autonomy.

Ambiguity

A situation in which available project context permits more than one plausible interpretation of desired behavior.

Corrective Rework

Follow-up work required because an implementation did not satisfy the intended requirement or produced an unintended defect.

Iterative Refinement

Follow-up work that improves a functioning implementation after use reveals a preferable product direction.

Provenance

The research layer documenting evidence related to origin, first known use, historical development, sources, and research status. It is distinct from the evidence required merely to understand a term's current meaning.

Appendix G — Source Notes and Methodology Details

The report distinguishes the frozen September 18 snapshot from present-day repository state, historical development evidence, development conversations, direct analytics, and later interpretation. The snapshot baseline is main after PR #133 (merge commit 4b31946), while current main remains authoritative for EpochLex after the reporting cutoff. This keeps later changes from being silently folded back into the first observation period.

Source Hierarchy

Priority	Use
1. Snapshot baseline / current main	For this edition, September 18 snapshot claims are frozen to main after PR #133 (merge commit 4b31946). Current main remains authoritative for EpochLex after the report cutoff.
2. Repository history	Pull requests, issues, commits, and prior versions used to reconstruct changes over time.
3. Development conversations	Used to establish intent, rejected alternatives, human judgments, testing feedback, and ambiguity not visible in Git history.
4. Direct usage analytics	Used for tokens and tool/model activity when the record can be attributed specifically to EpochLex.
5. Derived analysis	Metrics and interpretations calculated from the above sources and identified as analysis rather than raw evidence.

Evidence Handling

Earlier material does not override a later repository decision. Snapshot-state claims are checked against the frozen baseline; present-day claims are checked against current main; and historical pull requests, issues, and conversations explain how the project moved between those states. Unresolved values remain unresolved, and derived measurements are labeled as measured, reconstructed/estimated, or interpretive.

Citation Convention

Pull requests and issues are identified directly in the narrative because they function as evidence anchors within the public development record. Appendix A provides clickable links for the principal repository artifacts and maps the major findings to those anchors. Development conversations are identified by date and topic because they are project records rather than public repository URLs; external research introduced beyond project artifacts should use conventional source attribution appropriate to the source and claim.

Measurement Labels

Label	Meaning
Measured	Directly observed or counted from repository state, direct analytics, or another auditable record.
Reconstructed / Estimated	Calculated from incomplete historical records using an explicit method and stated uncertainty. If the source basis is insufficient, the value remains unavailable.
Interpretive	A case-study conclusion or conceptual model derived from the evidence rather than a directly measured quantity.



EpochLex is an open-source, searchable, data-driven reference work documenting terminology of the AI era.

The project continues to evolve as both a Living Dictionary and an ongoing experiment in human-directed AI-assisted development.

Development model:

Vibe coded • Human-directed • AI-assisted • Human-reviewed

Repository:

jeffthomasiii/dictionary-of-the-ai-era

Public project:

[EpochLex — Dictionary of the AI Era](#)

The September 18, 2026 report establishes the baseline for the next observation period rather than a final assessment of the project or the technology used to build it.